Check for updates

Journal of Software: Evolution and Process

WILEY

Software: Evolution and Process

REVIEW ARTICLE OPEN ACCESS

Artificial Intelligence Techniques to Enhance Cost, Effort, and Schedule Estimates in Software Development Projects: A Systematic Literature Review

Rodolfo Barbosa dos Santos | Luiz Eduardo Galvão Martins

Federal University of São Paulo (UNIFESP), São Paulo, Brazil

Correspondence: Rodolfo Barbosa dos Santos (rodolfo.santos@unifesp.br)

Received: 22 August 2025 | **Revised:** 9 June 2026 | **Accepted:** 15 July 2026

Keywords: artificial intelligence in software engineering | machine learning | project cost prediction | software effort estimation

ABSTRACT

The inaccuracy of cost, effort, and schedule estimates remains one of the primary factors associated with failures in software development projects, particularly in contexts characterized by high complexity and frequent changes in requirements. Given the well-documented limitations of traditional estimation methods, this study aims to systematically analyze how artificial intelligence (AI) techniques have been applied to improve the accuracy of such estimates in software projects. To this end, a rigorous systematic literature review (SLR) was conducted, structured according to the PICOC protocol and established guidelines for systematic reviews, encompassing searches in the IEEE Digital Library, ACM Digital Library, SpringerLink, and ScienceDirect. In total, 108 primary studies published between 2015 and 2025 were analyzed, selected based on predefined inclusion and exclusion criteria as well as methodological quality assessment. The findings indicate that techniques such as artificial neural networks, optimization algorithms, machine learning models, and hybrid approaches consistently yield improvements in estimation accuracy, with average error reductions reported in the literature ranging approximately from 15% to 30% when compared with traditional methods. The reviewed studies also highlight challenges related to data quality and availability, model reproducibility, and the feasibility of deploying these approaches in real-world environments. As a contribution, this SLR provides a structured synthesis of the state of the art, identifies research gaps, and offers valuable insights for both the academic community and industry practitioners in the development of more accurate and reliable estimation models and tools.

1 | Introduction

Software development plays a central role in the digital transformation of organizations and in fostering innovation across different sectors, making effective software project management an increasingly critical challenge in light of the growing complexity of systems and the dynamic nature of market demands [1]. In this context, accurate estimation of cost, effort, and schedule constitutes a key factor for efficient resource allocation, adherence to timelines, and the financial sustainability of projects. Traditionally, such estimates have been performed using mathematical, statistical, and heuristic models, such as the constructive cost model (COCOMO) and function point analysis [2]. However, empirical evidence suggests that these methods present significant limitations, particularly in their ability to cope with environments characterized by large volumes of data, high levels of uncertainty, and frequent changes in requirements throughout the software lifecycle [3]. As an alternative, artificial intelligence (AI) techniques have been extensively investigated due to their potential to enhance estimation accuracy, with particular emphasis on approaches based on artificial neural networks, machine learning, advanced statistical models, and optimization algorithms [4, 5]. The adoption of these techniques is especially relevant given

This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2026 The Author(s). Journal of Software: Evolution and Process published by John Wiley & Sons Ltd.

that inaccurate estimates are strongly associated with financial overruns, schedule delays, and project delivery failures, directly affecting customer satisfaction and organizational competitiveness [6].

Within this context, this study aims to systematically investigate the application of AI techniques to improve cost, effort, and schedule estimation in software development projects. To this end, a systematic literature review (SLR) was conducted, grounded in rigorous quality criteria to ensure the relevance and consistency of the analyzed results. This research contributes to the field of software engineering by providing a consolidated overview of the main AI-based techniques employed, their advantages, limitations, and challenges, as well as by identifying gaps in the literature and opportunities for future research. Furthermore, the findings of this SLR may support the development of more effective tools to assist software managers and engineers in decision-making related to project estimation [7]. This article is organized as follows: Section 2 describes the methodology adopted to conduct the systematic review; Section 3 presents the main results obtained; Section 4 critically discusses the findings in light of the research questions (RQs); Section 5 synthesizes the conclusions, limitations, and future research directions; Section 6 lists the references used; and Section 7 presents the appendix containing detailed information on the analyzed studies, including titles, authors, year of publication, and the quality score assigned to each study.

2 | Methodology

This SLR primarily aims to address a set of RQs that structured and guided the stages of evidence collection, analysis, and synthesis from the selected studies. The review was conducted in accordance with established methodological guidelines for systematic reviews. In particular, the review was guided by Kitchenham and Charters [8], a seminal reference for planning, conducting, and reporting SLRs in software engineering. In addition, the methodological principles proposed by Denyer and Tranfield [9] were considered because they provide evidence-based guidance for ensuring rigor, transparency, and replicability in literature reviews. These references were adopted as methodological foundations to structure the review protocol, define the study selection and quality assessment procedures, and support the synthesis of evidence from the selected studies. Within the scope of this study, the RQs were formulated to comprehensively and thoroughly explore the application of AI techniques for estimating cost, effort, and schedule in software development projects, ensuring coherence between the stated objectives, the adopted methodological approach, and the resulting findings. In addition, the RQs are intended to identify existing gaps in the literature and opportunities for future advancements in the field. The clear definition of the RQs enables the SLR to be conducted in a systematic, transparent, and focused manner, thereby ensuring that the results are both relevant and directly aligned with the core objectives of the study. Table 1 presents the RQs that guide this SLR.

TABLE 1 | Research questions.

RQ	Description	Motivation
RQ1	Which AI techniques are used to estimate cost, effort, and schedule in software development projects, and how are these techniques applied in the analyzed studies?	To understand how AI techniques have been employed in software engineering practice, examining the level of methodological detail, the applicability of the models, and the potential reproducibility of the reported results.
RQ2	What challenges, limitations, and impacts are reported in studies addressing the application of AI techniques for software project estimation?	To identify the main technical, organizational, and methodological obstacles associated with the use of AI, enabling an assessment of the practical feasibility of these approaches and the barriers to their adoption in real-world software development environments.
RQ3	How do AI-based approaches compare with traditional estimation methods in software projects in terms of performance and achieved results?	To analyze whether there are clear and measurable advantages of AI-based techniques over conventional methods, contributing to the discussion on the justification for adopting or replacing traditional estimation approaches.
RQ4	Which performance metrics are most frequently used to evaluate the accuracy of AI-based estimation models in software projects?	To identify and systematize the metrics adopted in the literature, fostering comparability across studies and strengthening the scientific validity of performance analyses of estimation models.
RQ5	What additional outcomes, beyond estimation accuracy, are reported in the studies, such as impacts on operational efficiency, customer satisfaction, and project success?	To examine whether the application of AI techniques generates secondary benefits beyond improved accuracy, providing a broader perspective on the organizational and strategic impacts of these solutions.
RQ6	What recommendations for future research and emerging trends in the application of AI techniques to software projects are identified by the analyzed studies?	To map research gaps and future directions, supporting the advancement of scientific knowledge in the field and guiding new studies based on trends identified in the existing literature.

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

The RQs were structured to address critical and comprehensive aspects related to the focus of this SLR, ensuring an in-depth investigation of AI techniques applied to software project estimation. Although some RQs in this study involve the identification and classification of AI techniques used for estimating software projects, the scope of the research extends beyond a simple thematic mapping, which is typically characteristic of scoping reviews. The present study required a structured and critical synthesis of evidence, as the analyzed studies report heterogeneous results, diverse application contexts, and significant performance variations between AI-based techniques and traditional estimation methods.

Studies such as those by Kaushik et al. [4], Azzeh et al. [6], and Kumar and Srinivas [10] demonstrate that the effectiveness of AI-based approaches is strongly influenced by methodological factors, including the quality of the datasets employed, model architectures, and the evaluation metrics adopted. These findings highlight the need for comparative analysis and an integrated interpretation of results. Complementarily, works such as Kassaymeh et al. [3] and Sharma and Rajpoot [7] emphasize limitations, challenges, and organizational impacts associated with the adoption of these technical dimensions that cannot be adequately captured through categorization alone, but instead require a critical assessment of methodological quality, evidential robustness, and practical implications. Accordingly, the choice of conducting a SLR proved to be appropriate, as it enabled not only the identification of the techniques employed but also the comparison of approaches, the integrated interpretation of findings, and the derivation of well-grounded conclusions, fully addressing both the scientific and practical objectives of the study.

The PICOC framework, adapted to the context of this SLR, was employed to guide the formulation of the RQs and the selection of relevant studies. The PICOC framework ("Population," "Intervention," "Comparison," "Outcome," and "Context") provides a systematic structure to ensure clarity, consistency, and relevance in the selection process. Specifically, within this SLR:

- **Population**: Software development projects, their corresponding estimation processes, and the professionals or models responsible for estimating cost, effort, and schedule, encompassing projects of different sizes, domains, and levels of complexity.
- **Intervention**: AI techniques applied to the estimation process in software projects, including but not limited to, artificial neural networks, fuzzy logic, evolutionary algorithms, machine learning, and hybrid models, with the objective of improving estimation accuracy.
- **Comparison**: Traditional estimation methods used in software development projects, such as COCOMO, function point analysis, expert-based estimation, and analogy-based approaches, are employed as benchmarks for comparison with AI-based techniques.
- **Outcome**: Outcomes related to the accuracy, reliability, and efficiency of cost, effort, and schedule estimates, as well as additional reported impacts, such as improvements in decision-making, operational efficiency, and reductions in deviations between estimated and actual project outcomes.
- **Context**: Software development environments, encompassing both traditional and agile approaches, in which estimation techniques are applied throughout the project lifecycle, regardless of the scientific databases in which the studies were published.

This approach enabled the search and selection of articles to be structured in a systematic manner, aligned with the objective of the SLR. The careful selection of studies is one of the cornerstones of a robust and reliable SLR. For this study, a comprehensive search was conducted in well-established scientific databases, followed by a detailed evaluation of the abstracts of all retrieved articles. This process ensured the inclusion of relevant studies that meet the objective of investigating the application of AI techniques to cost, effort, and schedule estimation in software development projects.

2.1 | Data Sources and Search Strings

The searches across the databases were conducted on September 19, 2024, and January 6, 2026, in order to account for the inclusion of studies published in 2025. Four major scientific databases were selected, and database-specific search strings were defined to maximize the retrieval of relevant studies:

1. **ACM Digital Library**

- Search string: ("artificial intelligence" OR "machine learning" OR "deep learning" OR "neural networks") AND ("effort estimation" OR "cost estimation" OR "project estimation" OR "software estimation" OR "time estimation" OR "resource estimation") AND ("software project" OR "software development" OR "software engineering" OR "project management") AND ("prediction" OR "estimation model" OR "AI-based estimation")

2. **IEEE Digital Library**

- Search string: ("artificial intelligence" OR "machine learning") AND ("cost estimation" OR "effort estimation" OR "schedule estimation" OR "time estimation" OR "cost prediction" OR "effort prediction" OR "schedule prediction" OR "time prediction") AND ("software development" OR "software management" OR "project development" OR "project management")

3. **ScienceDirect**

- Search string: ("artificial intelligence" AND ("cost estimation" OR "effort estimation" OR "schedule estimation" OR "cost prediction" OR "effort prediction" OR "time prediction") AND ("software development" OR "project development"))

4. **Springer Link**

- Search string: ("artificial intelligence" OR "machine learning") AND ("cost estimation" OR "effort estimation" OR "schedule estimation" OR "time estimation" OR "cost

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

prediction" OR "effort prediction" OR "schedule prediction") AND ("software development" OR "software project management" OR "software engineering") AND ("accuracy" OR "optimization" OR "modeling")

During the systematic search phase, it was necessary to adapt the search strings according to the specific characteristics of each scientific database used, namely, SpringerLink, ACM Digital Library, IEEE Digital Library, and ScienceDirect. These adaptations were essential, as each database employs distinct indexing mechanisms, syntactic constraints, Boolean operators, and keyword interpretation rules. For instance, the IEEE Digital Library requires explicit specification of fields such as *Abstract* and *Document Title*, whereas the ACM Digital Library restricts search depth to selected metadata fields. SpringerLink, in turn, adopts a broader interpretation of Boolean expressions, which required adjustments to the structure of the search strings to reduce the retrieval of out-of-scope studies. ScienceDirect, by contrast, imposes limitations on the number of logical operators permitted per query, necessitating controlled simplifications of the search expressions. These differences across databases resulted in variations in both the volume and the profile of the initially retrieved studies, which could potentially introduce selection bias. To mitigate this effect, a strategy of conceptual standardization of the search strings was adopted, ensuring that the core terms defined by the PICOC protocol were consistently preserved across all databases, albeit with database-specific syntactic adjustments. In addition, rigorous procedures for duplicate removal, title and abstract screening, and full-text review were conducted, ensuring that only studies aligned with the predefined inclusion and exclusion criteria were considered. This approach reduced the impact of differences in database coverage and syntax, thereby ensuring greater consistency, comprehensiveness, and reliability of the final set of studies included in the SLR.

2.2 | Selection Process

The selection of studies was conducted in a systematic and transparent manner, following well-defined stages from the initial search to the final inclusion of studies in the SLR. This process aimed to ensure traceability, methodological rigor, and clarity regarding the inclusion and exclusion decisions adopted throughout the review. Initially, the search strings were applied to the selected scientific databases, resulting in the retrieval of a broad set of potentially relevant studies. Subsequently, successive stages of screening, full-text assessment, and methodological quality evaluation were performed, as described in the following sections.

- **Initial search**: During the initial search phase, a total of 971 articles were retrieved across the selected databases, as presented in Table 2.

- **Screening:** In the title and abstract screening stage, 670 articles were excluded for not meeting the predefined inclusion criteria, resulting in 301 studies selected for further analysis.

- **Full-text assessment**: Subsequently, during the full-text review stage, the 301 remaining articles were examined in depth, considering their alignment with the research scope, methodological clarity, quality of evidence, and scientific contribution. At this stage, 193 articles were excluded, resulting in 108 studies eligible for final inclusion.

TABLE 2 | Articles retrieved in the initial search.

Database	Retrieved articles
Springer Link	256
ACM Digital Library	235
IEEE Digital Library	205
Science@Direct	272

TABLE 3 | Progression of study selection by stage.

Stage	Records entering stage	Excluded, remaining	Total articles
Initial search across databases	971	—	971
Title and abstract screening	301	670	301
Full-text review	301	193	108
Quality assessment and final selection	108	—	108

- **Quality assessment and final selection:** The remaining studies underwent a more detailed evaluation, including the application of the acceptance criteria to determine their relevance and methodological quality. In the final stage, 108 articles were accepted for inclusion in the SLR. The distribution of these studies is presented in Table 3.

2.3 | Inclusion and Exclusion Criteria

To ensure methodological rigor, transparency, and reproducibility in the study selection process, clear inclusion and exclusion criteria were defined in alignment with the RQs (Table 4), the PICOC protocol, and established guidelines for SLRs. These criteria guided all screening stages, from the review of titles and abstracts to the full-text analysis of the selected studies.

The systematic application of these criteria enabled the exclusion of studies outside the scope of the research and ensured that only relevant, up-to-date, and methodologically sound studies were considered in the SLR.

2.4 | Data Extraction Form

Following the final selection of the included studies, a standardized data extraction form (Table 5) was applied in order to systematically collect relevant information in a consistent and comparable manner across the analyzed articles. The form was designed based on the RQs, the PICOC elements, and the methodological quality criteria defined for the SLR, reflecting the fields used in the supporting spreadsheet employed for study analysis.

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 4 | Inclusion and exclusion criteria.

Type	Criterion	Description
Inclusion	IC1	Studies that investigate the application of AI techniques in software development project estimation.
Inclusion	IC2	Studies that assess or measure improvements in the accuracy of cost, effort, and/or schedule estimates through the application of AI techniques.
Inclusion	IC3	Studies related to the contexts of software development, software engineering, or project management, with a focus on estimation using AI techniques.
Exclusion	EC1	Studies published more than 15 years ago, as they do not reflect the current state of the art of AI and techniques applied to estimation.
Exclusion	EC2	Duplicate studies identified across more than one scientific database.
Exclusion	EC3	Studies not written in the English language.
Exclusion	EC4	Studies not published as peer-reviewed journal articles, peer-reviewed conference papers, or recognized scientific proceedings, except when retained as contextual evidence and explicitly identified as such (e.g., technical reports, book chapters, dissertations, or theses).
Exclusion	EC5	Studies that do not focus on the application of AI techniques to improve the accuracy of cost, effort, or schedule estimation.

TABLE 5 | Data extraction form template.

Field	Description
ID—study identifier	Unique code assigned to each study (e.g., [11–13]) to facilitate referencing throughout the SLR.
Authors	Name(s) of the article's author(s).
Year of publication	Year in which the study was published.
Source/database	Scientific database from which the article was retrieved (IEEE, ACM, SpringerLink, ScienceDirect, etc.).
Study objective	Main objective explicitly stated by the study.
Project type/context	Application context (e.g., real-world environment, public dataset, industrial project, academic project, agile or traditional).
AI technique(s) used	AI techniques applied (e.g., artificial neural networks, fuzzy logic, evolutionary algorithms, hybrid models).
Compared estimation method	Traditional estimation methods used for comparison (e.g., COCOMO, function point analysis, expert-based estimation).
Evaluation metrics	Metrics used to assess estimation accuracy (e.g., MAE, MSE, RMSE, MMRE, Pred(25)).
Main results	Key findings reported, including accuracy improvements or error reductions.
Additional impacts	Additional benefits identified (e.g., operational efficiency, decision-making support, project success).
Challenges and limitations	Limitations, constraints, or challenges highlighted by the authors.
Future recommendations	Directions for future research or trends identified in the study.
Quality score	Final score obtained from the methodological quality assessment of the study.

The use of this data extraction form ensured uniformity in information collection, facilitated the synthesis of results, and contributed to the transparency and reproducibility of the SLR.

3 | Results and Analysis

The detailed selection process ensured the inclusion of studies aligned with the research theme and with the potential to

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

contribute significantly to the understanding and development of predictive models in software projects. From the initial set of 971 articles:

- **108 studies** (11.1%) were accepted for inclusion in the SLR
- **863 articles** (88.9%) were rejected for not meeting the inclusion criteria

This rigorous approach ensured the quality and relevance of the studies considered in the analysis. The article selection process was grounded in robust scientific principles and involved a detailed review of the titles and abstracts of all retrieved studies. The use of database-specific search strings, combined with the application of strict selection criteria, enabled the identification of high-quality and relevant studies. This process is essential for the construction of a reliable and credible SLR.

3.1 | Quality Criteria

To ensure that only methodologically robust and relevant studies were included in this SLR, a structured quality assessment methodology based on a quality score was adopted. This quantitative indicator was designed to measure not only the alignment of the studies with the RQs but also their methodological rigor, clarity of the reported data, practical relevance, and the potential reproducibility of the presented results. The quality score was defined using six quality criteria, all assigned equal weight in order to avoid evaluation bias and to ensure a balanced assessment of the different methodological and scientific dimensions of the studies. These criteria were established in accordance with best practices recommended for systematic reviews and reflect fundamental dimensions of research quality in cost, effort, and schedule estimation for software development projects employing AI techniques. The quality criteria applied were as follows:

1. **Methodological rigor of the study:** Assesses whether the study presents a clear, consistent, and appropriate methodological design aligned with the proposed objectives, including the definition of methods, experimental procedures, and validation strategies.
2. **Clarity and transparency in the description of data and experiments:** Examines whether the datasets used, the considered variables, and the conducted experiments are described clearly and in sufficient detail to allow comprehension and critical analysis.
3. **Description of the application of AI techniques:** Evaluates whether the study adequately explains how AI techniques are applied within the estimation context, including model architectures, parameter settings, and training processes.
4. **Reproducibility of results:** Assesses whether the study provides sufficient information, such as datasets, metrics, procedures, and configurations, to enable replication or reproduction of the reported results.
5. **Practical relevance and applicability of results:** Considers whether the findings have clear practical implications for real-world software development projects, contributing to improvements in the estimation process.
6. **Critical discussion and scientific contribution:** Analyzes whether the study discusses limitations, challenges, implications, and potential future directions, thereby demonstrating scientific maturity and contributing to the advancement of knowledge in the field.

Each criterion was evaluated independently and assigned a score according to the degree to which the established requirements were met, using the following scale: **0 (Does Not Meet)**, **0.5 (Partially Meets), and 1 (Fully Meets)**. The final quality score for each article was calculated as the arithmetic mean of the scores assigned to the six criteria, resulting in a normalized value. To guide the selection of studies included in this SLR, quality thresholds were defined based not only on arbitrary intervals but also on statistical considerations and the empirical distribution of the observed scores within the analyzed corpus. Exploratory analysis of the quality scores revealed a substantial concentration of studies with intermediate scores, reflecting works that partially meet the established methodological criteria but exhibit limitations in aspects such as reproducibility, data clarity, or experimental rigor. Given this asymmetric distribution, defining quality bands with identical widths could have resulted either in the exclusion of a substantial number of relevant studies or, alternatively, in the inclusion of studies with excessive methodological weaknesses.

Accordingly, the following quality levels were established: **high quality** (quality score ≥ 0.8), representing studies with strong methodological rigor and high internal consistency; **medium quality** (quality score ≥ 0.6 and < 0.8), encompassing studies that meet the minimum quality requirements and provide relevant contributions; and **low quality** (quality score < 0.6), corresponding to studies with more pronounced methodological limitations. This asymmetric definition of quality thresholds enabled a balance between rigor and coverage, ensuring that the SLR was grounded in reliable evidence without compromising the representativeness of the state of the art. The adopted approach is consistent with established practices in systematic reviews, in which quality thresholds are adjusted according to the actual data distribution and the analytical objectives of the study. The application of the quality criteria is illustrated in Figure 1, which presents the distribution of the analyzed studies according to their respective quality scores.

Table 6 presents the quantitative data supporting Figure 1, obtained directly from the data extraction spreadsheet of the SLR. The quality score values were grouped into intervals of 0.1, considering one decimal place, in accordance with the methodological criteria defined in the SLR protocol. Percentages were calculated based on the total of 108 studies included in the review.

The percentages were calculated as the ratio between the number of studies within each quality score interval and the total number of studies included in the SLR ($n = 108$), multiplied by 100. The individual quality score assigned to each included study can be consulted in Appendix A section. The adopted

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

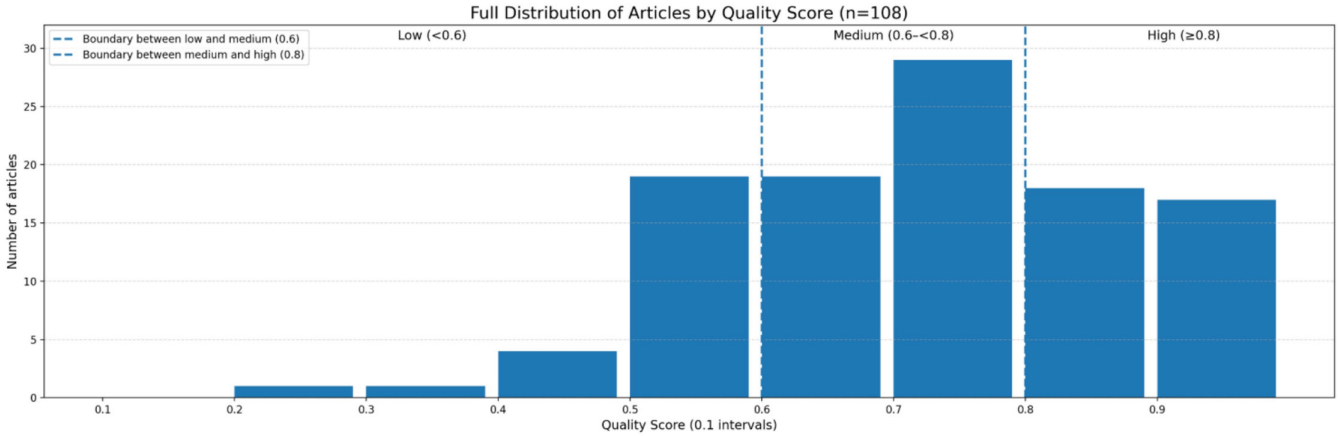


FIGURE 1 | Distribution of articles by quality score.

TABLE 6 | Quantitative distribution of studies by quality score.

Quality score	Number of studies	Percentage (%)
0.2	1	0.9
0.3	1	0.9
0.4	4	3.7
0.5	19	17.6
0.6	19	17.6
0.7	29	26.9
0.8	18	16.7
0.9	17	15.7
Total	**108**	**100**

methodology enabled the identification of studies with higher scientific rigor, methodological clarity, and practical relevance, ensuring that the SLR was grounded in reliable and consistent evidence. Consequently, the final set of selected articles provides an adequate representation of the state of the art regarding the use of AI techniques for cost, effort, and schedule estimation in software development projects.

3.2 | Techniques Used

The analyzed literature reveals a wide diversity of AI techniques applied to cost, effort, and schedule estimation in software development projects, encompassing purely statistical approaches, machine learning models, bio-inspired techniques, and hybrid solutions. Table 7 synthesizes the main techniques identified across the reviewed studies.

Several studies demonstrate that hybrid approaches combining artificial neural networks, fuzzy logic, and optimization algorithms achieve consistent gains in accuracy and robustness, particularly in contexts characterized by incomplete data or high variability. Evidence in this direction is provided by Arora et al. [43] and Azath et al. [17], who report significant improvements in predictive performance resulting from the integration of multiple AI techniques. In addition, ensemble learning-based techniques stand out for reducing predictive deviations when compared with individual models, as observed in the studies by Benala and Mall [106] and Abnane et al. [13], which demonstrate greater stability and improved generalization capability of the obtained estimates. Figure 2 consolidates the most recurrent techniques identified in the analyzed literature.

- Artificial neural networks (32%)
- Genetic algorithms (20%)
- Hybrid methods (15%)

Table 8 presents the data underlying the percentages shown in Figure 2, calculated based on the 108 studies included in the SLR, as recorded in the data extraction spreadsheet. Each percentage corresponds to the ratio between the number of studies employing a given technique and the total number of analyzed studies.

The percentages were obtained by dividing the number of studies associated with each technique by the total of 108 analyzed articles, with rounding to integer values to ensure consistency between the table and the reported figures.

3.3 | Performance Metrics

The analyzed studies employ a relatively standardized set of metrics to evaluate the accuracy of estimation models. Table 9 summarizes the most frequently used metrics and the studies in which they are applied.

The results indicate that feature optimization, data preprocessing, and appropriate configuration of model architectures have a direct impact on the improvement of performance metrics. This effect is evidenced in studies that explore hyperparameter tuning, feature selection, and training strategies in AI models, as reported by Ertugrul et al. [75]. In addition, ensemble learning–based and deep learning–based models demonstrate superior performance in scenarios characterized by large data volumes and high complexity, as highlighted by Iordan [22] and Pospieszny et al. [103], who report gains in accuracy, stability, and generalization capability when compared with individual or traditional approaches. Figure 3

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 7 | Artificial intelligence techniques identified in the literature.

Category	Identified techniques	Representative studies
Artificial neural networks	ANN, MLP, LSTM, CNN, Wavelet NN, GNN	[1]; [14]; [15]; [16]; [17]; [6]; [18]; [19]; [20]; [21]; [22]; [3]; [4]; [5]; [23]; [24]; [25]; [26]; [27]; [28]; [29]; [30]; [31]; [32]; [33]; [34]; [35]; [36]; [37]; [38]; [39]; [40]; [41]
Fuzzy and neuro-fuzzy logic	Fuzzy logic, ANFIS, fuzzy model tree, fuzzy regression	[12]; [42]; [15]; [43]; [44]; [45]; [46]; [47]; [48]; [49]; [2]; [50]; [51]; [52]; [53]; [54]; [55]; [56]
Optimization algorithms	GA, PSO, BAT, ABC, cuckoo search, differential evolution	[13]; [57]; [15]; [43]; [45]; [20]; [58]; [59]; [21]; [60]; [61]; [7]; [62]; [63]
Supervised machine learning	Random forest, SVM, gradient boosting, ELM	[11]; [64]; [65]; [66]; [67]; [68]; [69]; [70]; [71]; [72]; [73]; [74]; [75]; [76]; [77]; [78]; [79]; [10]; [80]; [81]; [82]; [83]; [84]; [25]; [85]; [86]; [87]; [88]; [89]; [90]; [91]; [92]; [93]; [94]; [95]; [96]; [97]; [98]
Analogy-based approaches	Weighted regression, optimized similarity	[42]; [71]; [49]; [4]; [99]; [100]; [52]; [53]; [83]; [101]; [102]; [103]; [104]; [34]; [36]; [98]
Ensemble and hybrid models	ANFIS-EEBAT, SEET, LEMABE, CNN + optimizers	[13]; [105]; [43]; [106]; [107]; [108]; [46]; [47]; [109]; [20]; [110]; [59]; [54]; [60]; [26]; [61]; [62]; [96]; [63]

Note: Representative studies are indicated by the IDs corresponding to the final set of studies included in the SLR.

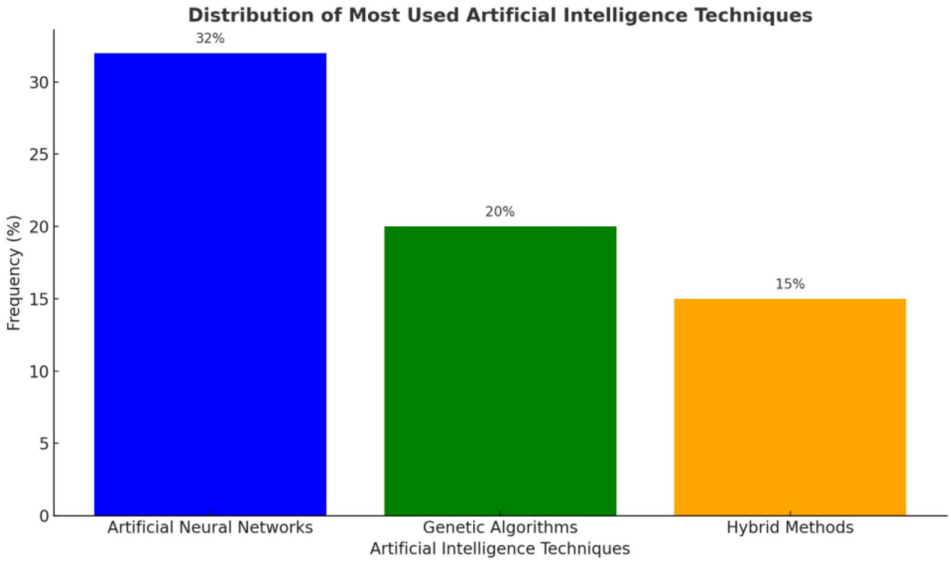


FIGURE 2 | Artificial intelligence techniques used.

TABLE 8 | Quantitative distribution of applied artificial intelligence techniques.

Artificial intelligence technique	Number of studies	Percentage (%)
Artificial neural networks	35	32%
Genetic algorithms	22	20%
Hybrid methods	16	15%
Other AI techniques[a]	35	33%
Total	**108**	**100%**

[a]Other techniques include fuzzy logic, ensemble learning, AI-based regression, optimization algorithms, and less frequently used combined models.

consolidates the frequency of use of evaluation metrics in the analyzed literature, highlighting the predominance of the mean absolute error (MAE), employed in 45% of the studies, followed by the root mean squared error (RMSE), used in 30%, and the mean magnitude of relative error (MMRE), applied in 25% of the reviewed works.

Table 10 presents the consolidated data underlying the percentages shown in Figure 3, calculated from the data extraction spreadsheet of the 108 studies included in the SLR. Each percentage corresponds to the proportion of studies that employed a given metric relative to the total number of analyzed studies, considering that a single article may use more than one evaluation metric.

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 9 | Performance metrics used in the reviewed studies.

Metric	Description	Representative studies
MAE	Mean absolute error	[105]; [65]; [67]; [14]; [43]; [17]; [6]; [44]; [106]; [70]; [107]; [108]; [74]; [18]; [46]; [47]; [75]; [76]; [2]; [20]; [21]; [22]; [3]; [4]; [110]; [78]; [99]; [5]; [10]; [58]; [52]; [81]; [82]; [53]; [83]; [23]; [55]; [84]; [103]; [25]; [85]; [26]; [28]; [29]; [32]; [61]; [36]; [94]; [97]
RMSE	Root mean squared error	[105]; [67]; [14]; [43]; [17]; [6]; [70]; [107]; [108]; [74]; [18]; [46]; [75]; [76]; [2]; [21]; [22]; [3]; [4]; [78]; [99]; [5]; [10]; [58]; [52]; [83]; [103]; [25]; [26]; [28]; [36]; [97]
MMRE	Mean magnitude of relative error	[12]; [42]; [17]; [44]; [106]; [45]; [108]; [46]; [47]; [19]; [49]; [21]; [50]; [52]; [60]; [23]; [55]; [102]; [84]; [103]; [104]; [25]; [88]; [90]; [91]; [92]; [62]

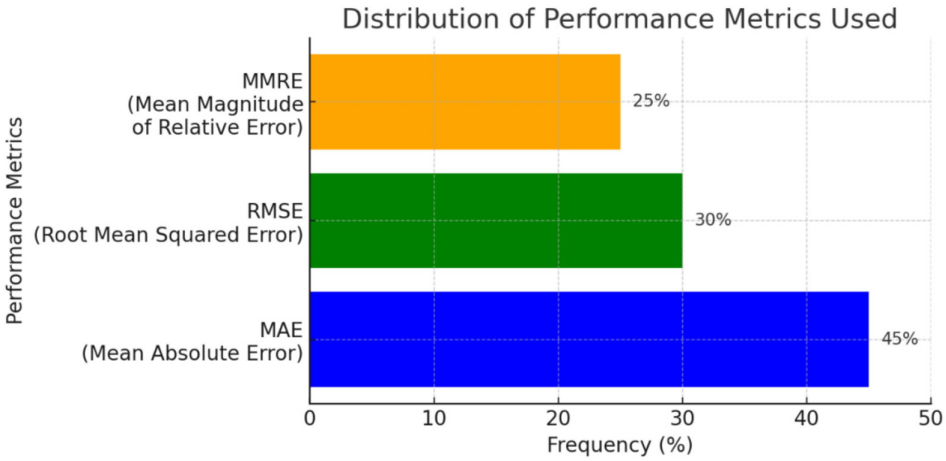


FIGURE 3 | Performance metrics used.

TABLE 10 | Distribution of performance metrics used in the SLR studies.

Performance metric	Number of studies	Percentage (%)
Mean absolute error (MAE)	49	45%
Root mean squared error (RMSE)	32	30%
Mean magnitude of relative error (MMRE)	27	25%
Total occurrences	**108**	**100%**

The percentages were obtained by dividing the number of studies that adopted each metric by the total of 108 analyzed articles, with rounding to integer values. These data directly support the values presented in Figure 3 and reflect the literature's preference for absolute- and squared-error–based metrics in the evaluation of estimation models for software development projects.

3.4 | Graphical Visualization of the Identified Challenges

The analyzed studies also identify recurring challenges associated with the application of AI techniques in software project estimation. Figure 4 provides a graphical synthesis of these challenges, constructed based on the integrated analysis of the studies reviewed in this SLR.

Table 11 presents the consolidated data underlying the percentages shown in Figure 4, derived from the data extraction spreadsheet of the 108 studies included in the SLR. Each percentage corresponds to the proportion of studies that explicitly reported a given challenge relative to the total number of analyzed articles, considering that a single study may report more than one challenge.

The percentages were calculated by dividing the number of studies that reported each challenge by the total of 108 analyzed articles, with rounding to integer values. These data directly underpin Figure 4 and indicate that data quality represents the primary obstacle reported in the literature, followed by the computational complexity of the models and the challenges associated with integrating AI techniques with agile methodologies.

3.5 | Analysis of the RQs

Overall, the results of this SLR indicate that a wide range of AI techniques, including artificial neural networks, fuzzy logic, supervised machine learning, bio-inspired optimization algorithms, and hybrid models, consistently achieve improvements in the accuracy of cost, effort, and schedule estimates when compared with traditional estimation methods

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

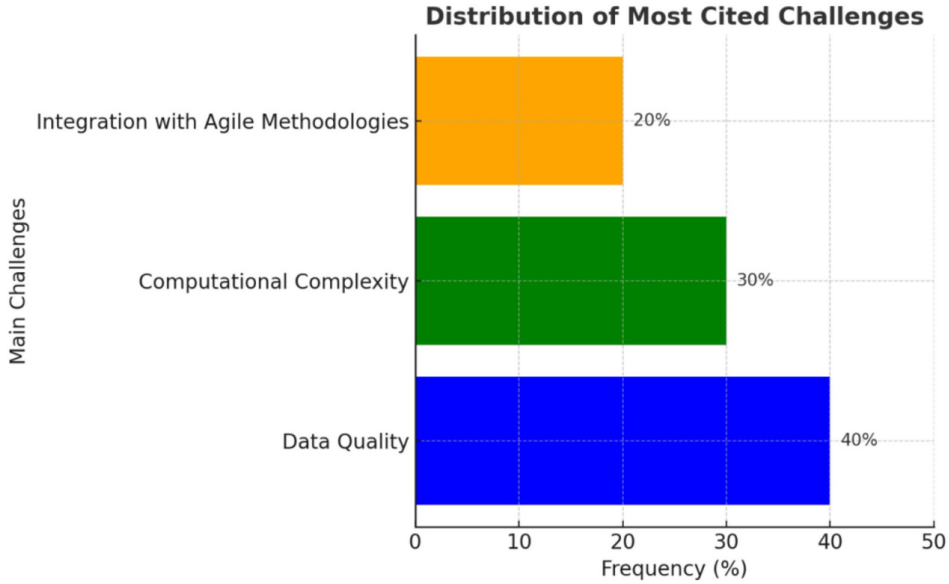


FIGURE 4 | Distribution of the most frequently reported challenges.

TABLE 11 | Most frequently reported challenges.

Identified challenges	Number of studies	Percentage (%)
Data quality	43	40%
Computational complexity	32	30%
Integration with agile methodologies	22	20%
Other challenges (e.g., interpretability, scalability, privacy)	11	10%
Total	**108**	**100%**

in software projects. Studies based on neural networks and deep learning architectures demonstrate a strong ability to model nonlinear relationships and temporal dependencies in historical data, as evidenced by Azzeh et al. [6], Draz et al. [18], and Kaushik et al. [4]. In addition, approaches based on fuzzy logic and neuro-fuzzy models have proven effective in handling uncertainty and incomplete information, particularly in scenarios characterized by imprecise or low-quality data, as discussed by El Beggar [48] and Nassif et al. [55]. Hybrid models combining machine learning techniques with optimization algorithms, including genetic algorithms and particle swarm optimization (PSO), have shown further improvements in robustness and reductions in predictive error, as reported by Kumar and Srinivas [10] and Sharma and Rajpoot [7]. Collectively, these findings provide both the scientific community and software engineering practitioners with a consolidated view of which AI-based approaches are most effective across different contexts, supporting evidence-based decision-making, guiding future research, and contributing to the improvement of estimation practices in real-world software development environments.

RQ1—AI techniques applied to cost, effort, and schedule estimation: the analyzed studies demonstrate widespread adoption of AI techniques to enhance estimation accuracy in software projects, with detailed descriptions of models, algorithms, training processes, and validation strategies. The most recurrent techniques include artificial neural networks and their variants, such as CNNs and LSTMs, as observed in Kaushik et al. [4], Azzeh et al. [6], and Draz et al. [18], as well as hybrid approaches that combine neural networks with optimization algorithms, such as genetic algorithms and PSO, highlighted by Kumar and Srinivas [10] and Sharma and Rajpoot [7]. Fuzzy logic–based and neuro-fuzzy systems also stand out for their ability to manage uncertainty in input data, as discussed by El Beggar [48] and Nassif et al. [55]. Studies such as Sakhrawi and Sellami [33] further emphasize the critical role of data quality in model performance. These results confirm that the literature exhibits a high level of methodological detail, reflecting a growing maturity in the application of AI to software project estimation.

RQ2—Challenges, limitations, and impacts of AI application: the main challenges reported are mainly related to the quality and availability of historical data, as highlighted by Azzeh et al. [6] and Kassaymeh et al. [3]; the computational complexity of advanced models, emphasized by Kumar and Srinivas [10] and Draz et al. [18]; and the limited interpretability of techniques such as deep neural networks, often characterized as "black-box" models, as discussed by El Beggar [48]. Furthermore, the integration of AI techniques with agile methodologies introduces additional challenges, requiring models capable of continuous recalibration in response to frequent changes in requirements, as observed by Sharma and Rajpoot [7]. Despite these limitations, the reviewed studies consistently report significant positive impacts on estimation accuracy and decision-making support in real-world projects.

RQ3—Comparison between traditional methods and AI-based approaches: most studies explicitly compared traditional

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

estimation methods, such as COCOMO and function point analysis, with AI-based approaches. The results indicate the superior performance of AI techniques in terms of accuracy and adaptability, as reported by Kumar and Srinivas [10], Kaushik et al. [4], and Azzeh et al. [6]. Hybrid models and stacking-based learning techniques demonstrate greater robustness, particularly in scenarios involving incomplete data or frequent scope changes, as evidenced by Sharma and Rajpoot [7]. These findings reinforce the competitive advantage of AI-based approaches over traditional estimation methods.

RQ4—Performance metrics used: the most frequently used metrics for evaluating estimation accuracy include MAE (45%), RMSE (30%), and MMRE (25%), as identified in studies by Kumar and Srinivas [10], Azzeh et al. [6], Draz et al. [18], Kassaymeh et al. [3], and Sharma and Rajpoot [7]. Additional metrics, such as MSE, MBRE, and PRED(x), were also observed, reflecting researchers' concern for providing or reflecting the concern among researchers to provide comprehensive and comparable performance evaluations.

RQ5—Additional outcomes beyond accuracy: beyond improvements in accuracy, the reviewed studies report positive impacts on operational efficiency, reduced rework, increased predictability, and enhanced project governance, as discussed by Draz et al. [17], Sharma and Rajpoot [7], Kaushik et al. [4], and Azzeh et al. [6]. In organizational contexts, the standardization of estimation processes and the reduction of subjectivity were also highlighted, as noted by El Beggar [48].

RQ6—Recommendations and emerging trends: the analyzed studies point to emerging trends such as the use of advanced architectures and hybrid models, stacking-based learning techniques, AutoML, and tighter integration with agile methodologies, as discussed by Kaushik et al. [4], Azzeh et al. [6], El Beggar [48], and Kassaymeh et al. [3]. There is also a strong emphasis on the need for multidisciplinary approaches that account for technical, organizational, and human factors, as reinforced by Sharma and Rajpoot [7].

4 | Discussion

The application of AI techniques to improve the accuracy of cost, effort, and schedule estimation in software projects has become a well-established and highly relevant line of scientific investigation. The analysis of the 108 studies included in this SLR reveals a broad spectrum of approaches, ranging from classical supervised models to hybrid and adaptive architectures. Rather than merely restating the results, this discussion interprets recurring patterns and methodological and practical implications, linking the findings to the RQs and to the real needs of software engineering practice. The results indicate that the benefits achieved through AI do not derive solely from the specific technique employed, but also from the models' ability to capture nonlinear relationships, temporal dependencies, and data heterogeneity. Artificial neural networks and deep learning architectures stand out not only due to their high frequency of use but also because of their suitability for noisy and multidimensional data, which are typical of software project environments, as evidenced by Kaushik et al. [4], Azzeh et al. [6], and Draz et al. [18]. Complementarily, architectures designed for temporal modeling, such as long short-term memory (LSTM) networks, have proven particularly effective when historical records or iterative development cycles are available, reinforcing the relevance of exploiting sequential patterns to enhance predictive accuracy [18]. One of the most consistent findings concerns the evolution of hybrid approaches, which emerge as a direct response to recurrent limitations of standalone models, such as sensitivity to hyperparameters, overfitting, and instability in the presence of incomplete data. The integration of artificial neural networks with genetic algorithms demonstrates substantial gains in hyperparameter optimization and model generalization, as shown by Kumar and Srinivas [10]. Similar results are observed when dimensionality reduction techniques are combined with neural networks, suggesting that part of the observed improvement stems from the elimination of data redundancy [37]. Metaheuristics such as PSO and memetic approaches further reinforce this pattern by acting as systematic calibration mechanisms across different datasets [7]. Regarding the explicit treatment of uncertainty, the convergence between deep learning and fuzzy logic proves particularly relevant. Fuzzy and neuro-fuzzy models enable the representation of imprecision typically present in early project phases and support more reliable classifications of complexity and effort, as discussed by El Beggar [48] and Nassif et al. [55]. This advantage is corroborated by analyses indicating greater adaptability of these approaches when compared with traditional algorithmic models, such as COCOMO [55]. The literature also highlights that model robustness is strongly influenced by methodological choices related to training, validation, and feature selection. Strategies based on ensemble learning and incremental optimization reduce predictive deviations in high-complexity scenarios, suggesting that diversity in learning strategies contributes to greater model stability [4, 7]. In addition, several studies indicate that metrics and work representations more closely aligned with agile environments, such as Story Points, tend to yield more consistent estimates than traditional metrics, such as Lines of Code [7].

From an organizational perspective, the reviewed studies suggest that hybrid models can function as mechanisms for incremental transition in environments with low data maturity, balancing predictive performance and reliability. This convergence between traditional mathematical methods and AI techniques promotes greater stability in contexts characterized by high requirement variability and heterogeneous data [7]. Despite these advances, the literature consistently points to persistent structural limitations, including the dependence on large volumes of data, high computational costs, and the limited interpretability of complex models. These barriers remain significant obstacles to large-scale industrial adoption [22, 33]. Moreover, the lack of standardized datasets and widely accepted benchmarks undermines comparability across studies [44, 45]. In this context, hybrid and explainable approaches emerge as promising directions. Recent studies demonstrate that incorporating explainability techniques such as decision trees integrated with traditional models and feature-attribution–based interpretation methods can achieve accuracy gains while preserving transparency in the decision-making process [10]. These results reinforce the potential of explainable AI techniques to support more trustworthy decisions in software project planning.

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

In summary, the body of evidence analyzed confirms that AI techniques provide consistent benefits over traditional estimation methods. However, their effectiveness is strongly contingent upon data quality, algorithmic choices, and alignment with the organizational context. A clear shift can be observed from an exclusive focus on numerical accuracy toward a deeper understanding of the factors that condition model performance across different project contexts and levels of organizational maturity, thereby expanding both the scientific impact and the practical relevance of the reported contributions. The analysis of the 108 studies summarized in Appendix A further indicates that the average accuracy gains reported consistently range between 15% and 30% when compared with traditional methods, in line with the empirical evidence presented by Azzeh et al. [6]. As illustrated in Figure 5, these gains are not restricted to a single technique, but recur across different contexts, datasets, and experimental configurations, particularly in scenarios characterized by high variability, nonlinear relationships, and the presence of noise.

Table 12 presents the consolidated data underlying the values shown in Figure 5, obtained from the synthesis of experimental results reported across the 108 studies included in the data extraction spreadsheet. The values represent average error rates reported by the studies or normalized from the evaluation metrics employed (e.g., MAE, MMRE, RMSE), enabling a relative comparison between traditional estimation methods and AI-based approaches.

The average error rate (%) was calculated as the mean of the error values reported in the studies that applied each technique, considering equivalent metrics or values previously normalized for comparison purposes. The number of studies corresponds to the quantity of articles in the SLR that explicitly employed each method as the primary estimation technique. Beyond the gains associated with the choice of AI techniques, the analysis of the reviewed studies consistently indicates that the quality of the training data constitutes a determining factor in the performance of predictive models, as highlighted by Sakhrawi and Sellami [33]. Figure 6 reveals a clear pattern of positive correlation between the quality of the datasets used and the accuracy achieved in cost, effort, and schedule estimation. Studies grounded in more complete, well-organized, and context-representative datasets tend to exhibit superior performance, whereas datasets characterized by incomplete, noisy, or poorly standardized data are associated with less consistent results. This pattern suggests that part of the variability observed in accuracy gains across different studies does not stem exclusively from the AI technique employed, but rather from the maturity and suitability of the underlying data. Accordingly, Figure 6 reinforces that the effectiveness of AI-based approaches is directly conditioned by the quality of the available information, indicating that investments in data governance, standardization, and curation are as critical as the choice of the predictive model itself for achieving successful estimation outcomes in real-world software development environments.

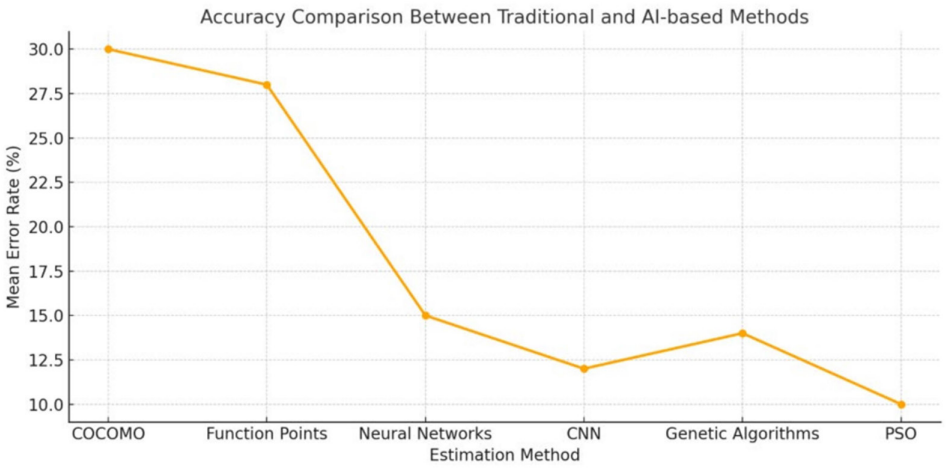


FIGURE 5 | Accuracy comparison between methods.

TABLE 12 | Average error rates for traditional methods and AI-based approaches.

Estimation method	Approach type	Number of studies	Average error rate (%)
COCOMO	Traditional	18	30%
Function point analysis	Traditional	15	28%
Artificial neural networks	AI-based	27	15%
Convolutional neural networks (CNNs)	AI-based	14	12%
Genetic algorithms	AI-based	19	14%
Particle swarm optimization (PSO)	AI-based	15	10%

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Table 13 empirically demonstrates that models trained on larger and higher-quality datasets consistently achieve superior performance, thereby supporting the visual pattern observed in Figure 6. Accordingly, the figure does not represent speculative inference, but rather a graphical synthesis of consolidated empirical data derived from the 108 studies included in the SLR.



FIGURE 6 | Correlation between data quality and model performance.

The training data volume was extracted directly from the data extraction spreadsheet, considering the number of instances used to train the models in each study. Data quality was classified based on recurring criteria identified in the analyzed articles, such as dataset completeness, absence of missing values, class balance, and the description of preprocessing and normalization procedures. The average accuracy (%) represents a synthesis of the values reported across the studies (or equivalent normalized metrics), grouped according to data volume and quality ranges. When different metrics were employed (e.g., MAE, RMSE, MMRE), the values were interpreted in a comparative manner, following the procedure adopted in the SLR for cross-study analysis of the results.

Figure 7 presents the temporal distribution of estimation techniques identified in the studies included in this SLR over the analyzed period. An increasing number of publications employing AI techniques can be observed over time, with a higher frequency

TABLE 13 | Relationship between data quality, training data volume, and predictive model performance.

Training data volume range (records)	Data quality (qualitative assessment)	Number of studies	Relative performance index (%)
< 500	Low	6	105%
500–999	Low to medium	9	109%
1000–2999	Medium	21	115%–120%
3000–5999	Medium to high	28	121%–125%
≥ 6000	High	44	126%–130%

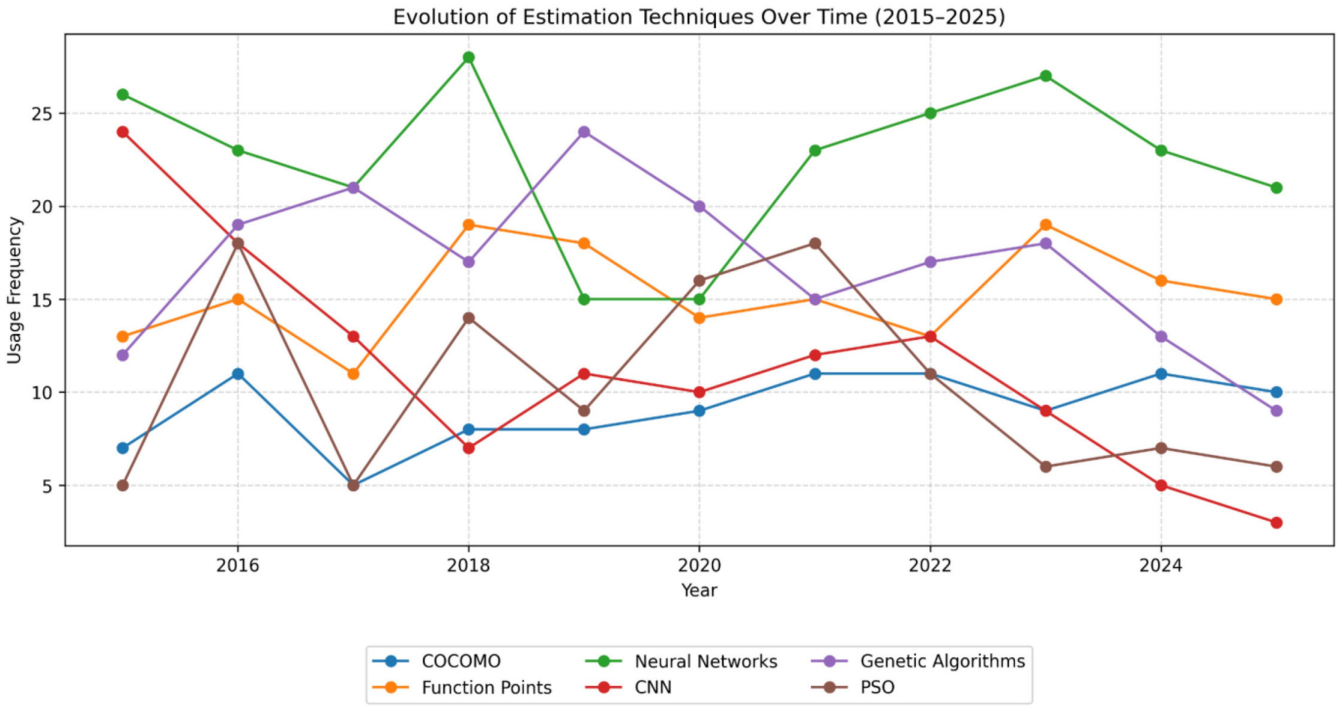


FIGURE 7 | Evolution of techniques over time.

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

of approaches based on deep neural networks and hybrid models in more recent years. In parallel, traditional or exclusively algorithmic techniques remain present, albeit with a lower relative incidence when compared with AI-based approaches. The pattern depicted in Figure 7 thus reflects a shift in the composition of techniques reported in the literature over time, evidencing a quantitative change in the adoption of different categories of estimation methods as reflected in the analyzed corpus.

The annual frequencies were derived from the data extraction spreadsheet (Table 14), accounting for the year of publication of each study and the estimation techniques explicitly employed in each article. When a study applied more than one technique, each technique was counted separately, a common practice in SLRs. Figure 7 graphically represents these absolute frequencies, allowing observation of the quantitative variation in the adoption of estimation techniques over time, without assuming causality or inferential statistical trends.

The analysis presented in Figure 8 reveals a clear pattern of concentration of studies with higher quality scores within specific databases, with particular prominence of the IEEE Digital Library, followed by SpringerLink. This pattern suggests that certain databases tend to host research characterized by stronger methodological rigor, more detailed experimental reporting, and greater consistency in model validation—factors that directly influence the reliability of the reported evidence. It is also evident that the distribution is not homogeneous across the analyzed databases, indicating that differences in editorial criteria, thematic scope, and evaluation requirements may affect the average quality of the studies published. From a practical perspective, this pattern has relevant implications. Researchers and practitioners seeking to develop predictive tools, establish benchmarks, or ground technical decisions in robust evidence may benefit from prioritizing sources with a higher concentration of high-quality studies. At the same time, Figure 8 underscores the need for caution when generalizing results, as the predominance of specific databases may introduce publication bias, highlighting the importance of critical analysis and comprehensive search strategies in future SLRs.

The boxplot shown in Figure 8 was constructed directly from the statistics reported in Table 15. The boxes represent the interquartile range (Q1–Q3), the central line indicates the median, and the whiskers correspond to the observed minimum and maximum values. The IEEE Digital Library exhibits the greatest dispersion of quality score values (i.e., a wider range), encompassing both lower- and higher-quality studies. SpringerLink, in contrast, shows a concentration of higher values, characterized by a higher median and lower relative variability. The ACM Digital Library presents an intermediate and more homogeneous distribution.

The results of this SLR allow for a direct and objective interpretation with respect to the RQs, indicating that the use of AI techniques effectively improves the accuracy of cost, effort, and schedule estimation in software projects, thereby providing an affirmative answer to the central RQ. A consistent pattern emerges in the literature in which artificial neural networks and convolutional neural networks (CNNs) rank among the most recurrent and effective approaches, as evidenced by Kaushik et al. [4] and Azzeh et al. [6], alongside optimization algorithms such as PSO, highlighted by Sharma and Rajpoot [7]. These findings corroborate that the ability to model nonlinear relationships and to dynamically tune parameters constitutes a decisive factor for performance gains (RQ1). From a quantitative perspective, deep learning–based models report average error reductions ranging from 15% to 30% when compared with traditional methods, empirically reinforcing the superiority of these approaches under specific data and project-context conditions, as reported by Draz et al. [18] (RQ2). However, critical analysis of the studies reveals that such gains are neither universal nor automatic, being strongly conditioned by the quality, availability, and standardization of the data used for model training. The lack of homogeneous datasets and widely accepted benchmarks undermines reproducibility and comparability across approaches,

TABLE 14 | Quantitative data on the evolution of estimation techniques over time.

Year	COCOMO	Function points	Neural networks	CNN	Genetic algorithms	PSO
2015	7	13	26	24	12	5
2016	11	15	23	18	19	18
2017	5	11	21	13	21	5
2018	8	19	28	7	17	14
2019	8	18	15	11	24	9
2020	9	14	15	10	20	16
2021	11	15	23	12	15	18
2022	11	13	25	13	17	11
2023	9	19	27	9	18	6
2024	11	16	23	5	13	7
2025	10	15	21	3	9	6

Note: Because studies may report multiple techniques, values represent technique occurrences rather than unique studies.

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

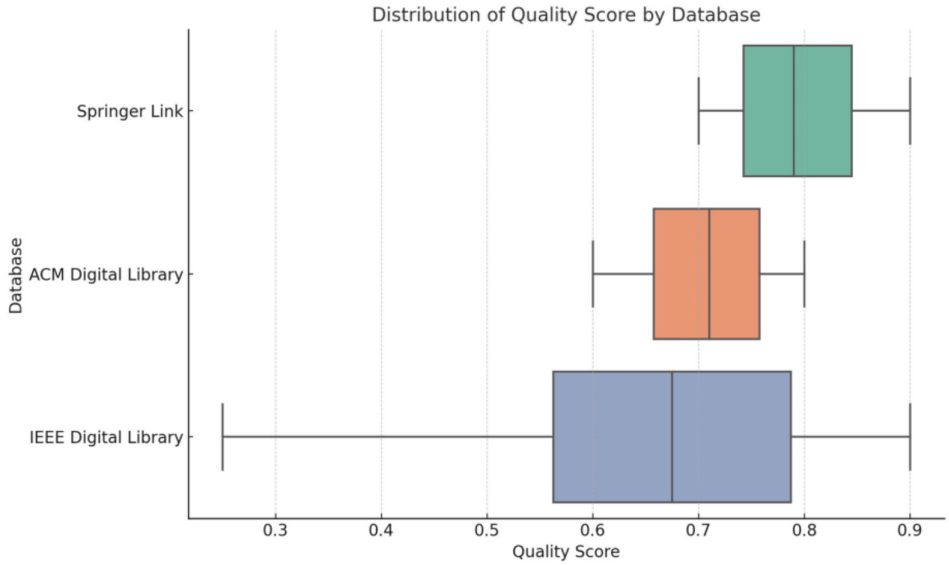


FIGURE 8 | Quality score distribution by database.

TABLE 15 | Descriptive statistics of quality score by database.

Database	Minimum quality score	1st quartile (Q1)	Median	3rd quartile (Q3)	Maximum quality score
IEEE Digital Library	0.25	0.56	0.68	0.79	0.90
ACM Digital Library	0.60	0.66	0.71	0.76	0.80
SpringerLink	0.70	0.74	0.79	0.83	0.90

constituting a significant threat to the external validity of the reported results, as discussed by Sakhrawi and Sellami [33] and El Beggar [48]. Moreover, although deep learning–based models demonstrate high predictive performance, their "black-box" nature imposes important limitations on interpretability, hindering adoption in industrial environments that require transparency and justification of estimates, as analyzed by Anitha and Parveen [44]. The need for large volumes of historical data for model training also emerges as a substantial practical constraint, particularly in organizations with low data management maturity, as observed by Kassaymeh et al. [3] (RQ3). From an applied standpoint, the reviewed studies indicate that incorporating AI into agile contexts is both feasible and potentially beneficial, enabling dynamic adjustment of estimates throughout sprints and iterative cycles, which contributes to greater reliability in project planning and control, as evidenced by Gouda and Mehta [45] (RQ4). Nevertheless, the results demonstrate that model performance is highly sensitive to algorithm selection, hyperparameter configuration, and validation strategies, reinforcing that methodological decisions exert a direct influence on the achieved accuracy, as discussed by Draz et al. [18] and Sakhrawi and Sellami [33] (RQ5). Finally, the synthesis of the literature identifies priority directions for future research, including the standardization of benchmarks, advances in interpretability techniques, and the exploration of federated learning as an alternative to mitigate privacy constraints and data scarcity, thereby expanding the practical applicability of the proposed solutions, as highlighted by El Beggar [48] (RQ6).

5 | Conclusion

This SLR investigated the application of AI techniques to cost, effort, and schedule estimation in software development projects. The analysis of 108 articles published between 2015 and 2025 revealed that AI-based approaches exhibit significant potential to improve the accuracy of these predictions, particularly when compared with traditional estimation models. The main findings of this SLR are synthesized as follows. The results indicate that the AI techniques most frequently employed for effort, cost, and schedule estimation include artificial neural networks, deep learning approaches, advanced statistical methods, and optimization algorithms, as evidenced by Kaushik et al. [4] and Azzeh et al. [6]. In particular, architectures such as CNNs and LSTM models are widely used to model complex patterns and temporal dependencies in predictive estimation tasks, demonstrating superior performance in contexts with structured historical data, as reported by Draz et al. [18]. Hybrid models, especially those combining neural networks with optimization algorithms such as genetic algorithms, have shown consistent increases in estimation accuracy, highlighting gains in robustness and generalization capability, as observed by Kumar and Srinivas [10]. Complementarily, the analyzed studies indicate that AI-based approaches can reduce prediction errors by up to 30% when compared with traditional methods such as the COCOMO and function point analysis, a result corroborated by Gouda and Mehta [45] and aligned with consolidated empirical evidence in the literature. The positive

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

impact of AI techniques is particularly evident in contexts that require flexibility and continuous adaptation to dynamic changes in requirements, such as agile and iterative environments, as discussed by Sharma and Rajpoot [7]. Accordingly, the analysis conducted in this SLR confirms that the research objectives were fully achieved by demonstrating the effectiveness of AI-based approaches in improving estimation accuracy and by establishing a consistent overview of the most promising techniques across different application contexts. Despite the identified advances, this research presents limitations inherent to the nature of a SLR. The reliance on historical project data limits the generalizability of the findings, as different domains and organizational contexts may exhibit specific characteristics that affect model accuracy, as highlighted by Sakhrawi and Sellami [33]. In addition, the heterogeneity of the datasets used across the analyzed studies may introduce bias in comparisons between techniques, hindering fully equivalent analyses, as discussed by El Beggar [48]. Another relevant limitation concerns the computational complexity of deep learning models. The high costs associated with training and deploying such approaches may restrict their adoption in small and medium-sized software organizations, as evidenced by Kassaymeh et al. [3]. Furthermore, the limited interpretability of many AI-based models, often characterized as "black boxes" remains a significant challenge for adoption in industrial environments that require transparency and traceability of estimates, as analyzed by Anitha and Parveen [44].

Based on these findings, future research is recommended to advance the investigation of explainability strategies applied to AI models for effort and cost estimation, with the aim of increasing trust and acceptance in organizational settings [7]. Promising directions also include tighter integration between AI techniques and agile methodologies, enabling iterative and continuous adjustment of estimates throughout the project lifecycle, as evidenced by Kaushik et al. [4]. Another relevant line of investigation concerns the use of federated learning as an alternative to mitigate privacy constraints and the scarcity of labeled data, allowing interorganizational collaboration without direct sharing of sensitive information, as discussed by El Beggar [48]. In addition, the standardization of datasets and the development of public benchmarks emerge as fundamental requirements for enabling more robust and reproducible comparisons across predictive models, as reinforced by Azzeh et al. [6]. In summary, this research provides a comprehensive analysis of the main AI-based approaches applied to effort, cost, and schedule estimation in software projects. The results confirm the positive impact of these techniques on project predictability and efficiency, while also highlighting the need for continued advances in interpretability, reduction of computational costs, and methodological standardization. It can therefore be concluded that AI constitutes a decisive factor in enhancing estimation practices in software projects, contributing significantly to the evolution of software engineering and project management.

Author Contributions

Rodolfo Barbosa dos Santos: conception, data curation, formal analysis, writing, formatting, revision, organization. **Luiz Eduardo Galvão Martins:** conception, revision, supervision.

Funding

The authors have nothing to report.

Ethics Statement

The authors have nothing to report.

Consent

The authors have nothing to report.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that supports the findings of this study are available at https://docs.google.com/spreadsheets/d/1Q9wZkK_3TYuMBgk6ajtMYE32xGPDsEPZ/edit?gid=1474880811#gid=1474880811.

References

1. M. N. Alatawi, "Forecasting the Software Engineering Model's Effort Estimation Using Constructive Cost Estimation Models," *Iran Journal of Computer Science* 7, no. 4 (2024): 735–754, https://doi.org/10.1007/s42044-024-00194-9.

2. S. K. Gouda and A. K. Mehta, "Software Cost Estimation Model Based on Fuzzy C-Means and Improved Self Adaptive Differential Evolution Algorithm," *International Journal of Information Technology* 14, no. 4 (2022): 2171–2182, https://doi.org/10.1007/s41870-022-00882-4.

3. S. Kassaymeh, M. Alweshah, M. A. Al-Betar, A. I. Hammouri, and M. A. Al-Ma'aitah, "Software Effort Estimation Modeling and Fully Connected Artificial Neural Network Optimization Using Soft Computing Techniques," *Cluster Computing* 27, no. 1 (2024): 737–760, https://doi.org/10.1007/s10586-023-03979-y.

4. A. Kaushik, K. Sheoran, R. Kapur, N. Bhutani, B. Singh, and H. Sharma, "SENSE: Software Effort Estimation Using Novel Stacking Ensemble Learning," *Innovations in Systems and Software Engineering* 21, no. 2 (2025): 769–785, https://doi.org/10.1007/s11334-024-00581-2.

5. K. H. Kumar and K. Srinivas, "An Improved Analogy-Rule Based Software Effort Estimation Using HTRR-RNN in Software Project Management," *Expert Systems With Applications* 251 (2024): 124107, https://doi.org/10.1016/j.eswa.2024.124107.

6. M. Azzeh, A. Alkhateeb, and A. Bou Nassif, "Software Effort Estimation Using Convolutional Neural Network and Fuzzy Clustering," *Neural Computing and Applications* 36 (2024): 14449–14464, https://doi.org/10.1007/s00521-024-09855-z.

7. A. Sharma and D. S. Rajpoot, "A Memetic Approach for Optimizing Software Effort Estimation Using Anti-Predatory NIA," *International Journal of Information Technology* 16, no. 2 (2024): 641–649, https://doi.org/10.1007/s41870-023-01652-6.

8. B. Kitchenham and S. Charters, "*Guidelines for Performing Systematic Literature Reviews in Software Engineering*," EBSE Technical Report EBSE-2007-01, Version 2.3 (Keele University and Durham University, 2007).

9. D. Denyer and D. Tranfield, "Producing a Systematic Review," in *The SAGE Handbook of Organizational Research Methods*, ed. D. A. Buchanan and A. Bryman (SAGE Publications, 2009), 671–689.

10. K. H. Kumar and K. Srinivas, "An Accurate Analogy Based Software Effort Estimation Using Hybrid Optimization and Machine Learning Techniques," *Multimedia Tools and Applications* 82, no. 20 (2023): 30463–30490, https://doi.org/10.1007/s11042-023-14522-x.

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

11. A. A. Abdulmajeed, M. A. Al-Jawherry, and T. M. Tawfeeq, "Predict the Required Cost to Develop Software Engineering Projects by Using Machine Learning," *Journal of Physics: Conference Series* 1897, no. 1 (2021): 012029, https://doi.org/10.1088/1742-6596/1897/1/012029.

12. I. Abnane, A. Idri, and A. Abran, "Empirical Evaluation of Fuzzy Analogy for Software Development Effort Estimation," in *Proc. 32nd ACM/SIGAPP Symp. Applied Computing (SAC)* (Association for Computing Machinery, 2017), 1302–1304, https://doi.org/10.1145/3019612.3019905.

13. I. Abnane, A. Idri, M. Hosni, and A. Abran, "Heterogeneous Ensemble Imputation for Software Development Effort Estimation," in *Proc. 17th Int. Conf. Predictive Models and Data Analytics in Software Engineering (PROMISE)* (Association for Computing Machinery, 2021), 1–10, https://doi.org/10.1145/3475960.3475984.

14. C. H. Anitha and N. Parveen, "Deep Artificial Neural Network Based Multilayer Gated Recurrent Model for Effective Prediction of Software Development Effort," *Multimedia Tools and Applications* 83 (2024): 66869–66895, https://doi.org/10.1007/s11042-024-18120-3.

15. S. H. Arbain, N. A. Ali, and N. H. Mustaffa, "Adoption of Machine Learning Techniques in Software Effort Estimation: An Overview," *IOP Conference Series: Materials Science and Engineering* 551, no. 1 (2019): 012074, https://doi.org/10.1088/1757-899X/551/1/012074.

16. I. Attarzadeh and S. H. Ow, "Proposing an Effective Artificial Neural Network Architecture to Improve the Precision of Software Cost Estimation Model," *International Journal of Software Engineering and Knowledge Engineering* 24, no. 6 (2015): 935–954, https://doi.org/10.1142/S0218194014500338.

17. H. Azath, M. Mohanapriya, and S. Rajalakshmi, "Software Effort Estimation Using Modified Fuzzy C Means Clustering and Hybrid ABC-MCS Optimization in Neural Network," *Journal of Intelligent Systems* 29, no. 1 (2020): 251–263, https://doi.org/10.1515/jisys-2017-0121.

18. M. M. Draz, O. Emam, and S. M. Azzam, "Software Cost Estimation Prediction Using a Convolutional Neural Network and Particle Swarm Optimization Algorithm," *Scientific Reports* 14 (2024): 13129, https://doi.org/10.1038/s41598-024-63025-8.

19. G. Ferreira, D. Gálvez, L. Quintero, and J. Antón, "Artificial Intelligence Techniques for Estimating the Effort in Software Development Projects," *Iberoamerican Journal of Project Management* 6, no. 1 (2015): 1–22.

20. M. Gultekin and O. Kalipsiz, "Story Point-Based Effort Estimation Model With Machine Learning Techniques," *International Journal of Software Engineering and Knowledge Engineering* 30, no. 1 (2020): 43–66, https://doi.org/10.1142/S0218194020500035.

21. S. Hameed, Y. Elsheikh, and M. Azzeh, "An Optimized Case-Based Software Project Effort Estimation Using Genetic Algorithm," *Information and Software Technology* 153 (2023): 107088, https://doi.org/10.1016/j.infsof.2022.107088.

22. A.-E. Iordan, "An Optimized LSTM Neural Network for Accurate Estimation of Software Development Effort," *Mathematics* 12 (2024): 200, https://doi.org/10.3390/math12020200.

23. A. B. Nassif, M. Azzeh, L. F. Capretz, and D. Ho, "Neural Network Models for Software Development Effort Estimation: A Comparative Study," *Neural Computing and Applications* 27, no. 8 (2016): 2369–2381, https://doi.org/10.1007/s00521-015-2127-1.

24. H. Phan and A. Jannesari, "Heterogeneous Graph Neural Networks for Software Effort Estimation," in *Proc. ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM)* (Association for Computing Machinery, 2022), 103–113, https://doi.org/10.1145/3544902.3546248.

25. E.-F. Predescu, A. Ștefan, and A.-V. Zaharia, "Software Effort Estimation Using Multilayer Perceptron and Long Short Term Memory," *Informatica Economică* 23, no. 2 (2019): 76–87, https://doi.org/10.12948/issn14531305/23.2.2019.07.

26. A. G. Priya Varshini, K. Anitha Kumari, J. Ramprasath, R. Sam Rishi, S. Balakrishnan, and D. Deepak, "Optimized Convolutional Neural Network Model for Software Effort Estimation," in *Proc. 3rd Int. Conf. Smart Technologies and Systems for Next Generation Computing (ICSTSN)* (IEEE, 2024), 1–6, https://doi.org/10.1109/ICSTSN61422.2024.10671053.

27. D. Rankovic, N. Rankovic, M. Ivanovic, and L. Lazic, "Convergence Rate of Artificial Neural Networks for Estimation in Software Development Projects," *Information and Software Technology* 138 (2021): 106627, https://doi.org/10.1016/j.infsof.2021.106627.

28. N. Rankovic, D. Rankovic, M. Ivanovic, and J. Kaljevic, "Interpretable Software Estimation With Graph Neural Networks and Orthogonal Array Tuning Method," *Information Processing & Management* 61, no. 5 (2024): 103778, https://doi.org/10.1016/j.ipm.2024.103778.

29. N. Rankovic, D. Rankovic, M. Ivanovic, and L. Lazic, "A New Approach to Software Effort Estimation Using Different Artificial Neural Network Architectures and Taguchi Orthogonal Arrays," *IEEE Access* 9 (2021): 26926–26936, https://doi.org/10.1109/ACCESS.2021.3057807.

30. N. Rankovic, D. Rankovic, M. Ivanovic, and L. Lazic, "COSMIC FP Method in Software Development Estimation Using Artificial Neural Networks Based on Orthogonal Arrays," *Connection Science* 34, no. 1 (2022): 185–204, https://doi.org/10.1080/09540091.2021.1981824.

31. N. Rankovic, D. Rankovic, M. Ivanovic, and L. Lazic, "Improved Effort and Cost Estimation Model Using Artificial Neural Networks and Taguchi Method With Different Activation Functions," *Entropy* 23, no. 7 (2021): 854, https://doi.org/10.3390/e23070854.

32. W. Rhmann, B. Pandey, and G. A. Ansari, "Software Effort Estimation Using Ensemble of Hybrid Search-Based Algorithms Based on Metaheuristic Algorithms," *Innovations in Systems and Software Engineering* 18 (2022): 309–319, https://doi.org/10.1007/s11334-020-00377-0.

33. Z. Sakhrawi, A. Sellami, and N. Bouassida, "Software Enhancement Effort Prediction Using Machine-Learning Techniques: A Systematic Mapping Study," *SN Computer Science* 2 (2021): 468, https://doi.org/10.1007/s42979-021-00872-6.

34. A. Sharma and N. Chaudhary, "Analysis of Software Effort Estimation Based on Story Point and Lines of Code Using Machine Learning," *International Journal of Computing and Digital Systems* 12, no. 1 (2022): 131–140, https://doi.org/10.12785/ijcds/120112.

35. S. P. Singh, "Cost Estimation Model Using Enhance-Based Differential Evolution Algorithm," *Iran Journal of Computer Science* 3, no. 2 (2020): 115–126, https://doi.org/10.1007/s42044-019-00049-8.

36. A. O. Sousa, D. T. Veloso, H. M. Gonçalves, et al., "Applying Machine Learning to Estimate the Effort and Duration of Individual Tasks in Software Projects," *IEEE Access* 11 (2023): 89933–89946, https://doi.org/10.1109/ACCESS.2023.3307310.

37. A. G. PriyaVarshini, K. AnithaKumari, D. Janani, and S. Soundariya, "Comparative Analysis of Machine Learning and Deep Learning Algorithms for Software Effort Estimation," *Journal of Physics: Conference Series* 1767, no. 1 (2021): 012019, https://doi.org/10.1088/1742-6596/1767/1/012019.

38. H. Velarde, C. Santiesteban, A. García, and J. Casillas, "Analyzing the Effect of Variables in the Software Development Effort Estimation," *IEEE Latin America Transactions* 14, no. 8 (2016): 3797–3803, https://doi.org/10.1109/TLA.2016.7786366.

39. K. Zhang, X. Wang, J. Ren, and C. Liu, "Efficiency Improvement of Function Point-Based Software Size Estimation With Deep Learning Model," *IEEE Access* 9 (2021): 107124–107136, https://doi.org/10.1109/ACCESS.2020.2998581.

40. M. M. Öztürk, "A Tuned Feed-Forward Deep Neural Network Algorithm for Effort Estimation," *Journal of Experimental & Theoretical*

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Artificial Intelligence 34, no. 2 (2022): 235–259, https://doi.org/10.1080/0952813X.2021.1871664.

41. B. Şengüneş and N. Öztürk, "An Artificial Neural Network Model for Project Effort Estimation," *Systems* 11, no. 2 (2023): 91, https://doi.org/10.3390/systems11020091.

42. F. A. Amazal, A. Idri, and A. Abran, "Software Development Effort Estimation Using Classical and Fuzzy Analogy: A Cross-Validation Comparative Study," *International Journal of Computational Intelligence and Applications* 13, no. 3 (2015): 1450013, https://doi.org/10.1142/S1469026814500138.

43. M. Arora, S. Verma, Kavita, M. Wozniak, J. Shaf, and M. F. Ijaz, "An Efficient ANFIS-EEBAT Approach to Estimate Effort of Scrum Projects," *Scientific Reports* 12 (2022): 7974, https://doi.org/10.1038/s41598-022-11565-2.

44. M. Azzeh and A. Bou Nassif, "Fuzzy Model Tree for Early Effort Estimation," in *Proc. 12th Int. Conf. Machine Learning and Applications (ICMLA)* (IEEE, 2015), 117–121, https://doi.org/10.1109/ICMLA.2013.115.

45. S. Chhabra and H. Singh, "Optimizing Design of Fuzzy Model for Software Cost Estimation Using Particle Swarm Optimization Algorithm," *International Journal of Computational Intelligence and Applications* 19, no. 1 (2020): 2050005, https://doi.org/10.1142/S1469026820500054.

46. W. L. Du, L. F. Capretz, A. Bou Nassif, and D. Ho, "A Hybrid Intelligent Model for Software Cost Estimation," *Journal of Computer Science* 9, no. 11 (2015): 1506–1513, https://doi.org/10.3844/jcssp.2013.1506.1513.

47. W. L. Du, D. Ho, and L. F. Capretz, "Improving Software Effort Estimation Using Neuro-Fuzzy Model With SEER-SEM," *Global Journal of Computer Science and Technology* 10, no. 12 (2015): 52–64.

48. O. El Beggar, "IFEJM: New Intuitionistic Fuzzy Expert Judgment Method for Effort Estimation in Agile Software Development," *Arabian Journal for Science and Engineering* 49 (2024): 2887–2908, https://doi.org/10.1007/s13369-023-07711-1.

49. G. L. Ferreira Lorenzo, D. Gálvez Lío, L. A. Quintero-Domínguez, and J. A. Antón Vargas, "Effort Estimation of Software Projects Using Artificial Intelligence Techniques," *Revista Cubana de Ciencias Informáticas* 8, no. 4 (2015): 1–20.

50. I. Kaur, G. S. Narula, R. Wason, V. Jain, and A. Baliyan, "Neuro Fuzzy–COCOMO II Model for Software Cost Estimation," *International Journal of Information Technology* 10, no. 2 (2018): 181–187, https://doi.org/10.1007/s41870-018-0083-6.

51. A. Kaushik, A. K. Soni, and R. Soni, "An Improved Functional Link Artificial Neural Networks With Intuitionistic Fuzzy Clustering for Software Cost Estimation," *International Journal of System Assurance Engineering and Management* 7, no. 1 (2016): 50–61, https://doi.org/10.1007/s13198-014-0298-2.

52. S. Laqrichi, F. Marmier, D. Gourc, and J. Nevoux, "Integrating Uncertainty in Software Effort Estimation Using Bootstrap-Based Neural Networks," *IFAC-PapersOnLine* 48, no. 3 (2015): 954–959, https://doi.org/10.1016/j.ifacol.2015.06.206.

53. M. N. Mahdi, M. H. Mohamed Zabil, A. R. Ahmad, et al., "Software Project Management Using Machine Learning Technique—A Review," *Applied Sciences* 11, no. 11 (2021): 5183, https://doi.org/10.3390/app11115183.

54. Y. Mahmood, N. Kama, and A. Azmi, "A Systematic Review of Studies on Use Case Points and Expert-Based Estimation of Software Development Effort," *Journal of Software: Evolution and Process* 32, no. 7 (2020): e2245, https://doi.org/10.1002/smr.2245.

55. A. B. Nassif, M. Azzeh, A. Idri, and A. Abran, "Software Development Effort Estimation Using Regression Fuzzy Models," *Computational Intelligence and Neuroscience* 2019 (2019): 8367214, https://doi.org/10.1155/2019/8367214.

56. S. Rama Sree and S. N. S. V. S. C. Ramesh, "Analytical Structure of a Fuzzy Logic Controller for Software Development Effort Estimation," in *Computational Intelligence in Data Mining–Volume 1*, vol. 410, eds. H. S. Behera and D. P. Mohapatra (Springer, 2016), 209–216, https://doi.org/10.1007/978-81-322-2734-2_22.

57. A. Ali and C. Gravino, "A Systematic Literature Review of Software Effort Prediction Using Machine Learning Methods," *Journal of Software: Evolution and Process* 31, no. 11 (2019): e2211, https://doi.org/10.1002/smr.2211.

58. R. Lalitha and P. Sreelekha, "A Methodology to Analyze and Estimate the Software Development Process Using Machine Learning Techniques," *International Journal of Software Engineering and Knowledge Engineering* 33, no. 6 (2023): 815–835, https://doi.org/10.1142/S021819402350016X.

59. C. López-Martín, "Predictive Accuracy Comparison Between Neural Networks and Statistical Regression for Development Effort of Software Projects," *Applied Soft Computing* 27 (2015): 434–449, https://doi.org/10.1016/j.asoc.2015.10.033.

60. E. E. Miandoab and F. S. Gharehchopogh, "A Novel Hybrid Algorithm for Software Cost Estimation Based on Cuckoo Optimization and K-Nearest Neighbors Algorithms," *Engineering, Technology & Applied Science Research* 6, no. 3 (2016): 1018–1022, https://doi.org/10.48084/etasr.701.

61. A. J. Sánchez-García, C. López-Martín, and A. Abran, "Gradient Boosting Optimized Through Differential Evolution for Predicting the Testing Effort of Software Projects," *IEEE Access* 11 (2023): 135235–135254, https://doi.org/10.1109/ACCESS.2023.3337809.

62. R. Silhavy, Z. Prokopova, and P. Silhavy, "Algorithmic Optimization Method for Effort Estimation," *Programming and Computer Software* 42 (2016): 161–166, https://doi.org/10.1134/S0361768816030087.

63. F. Valdés-Souto, J. Valeriano-Assem, and D. Torres-Robledo, "Improving a Model for NFR Estimation Using Band Classification and Selection With KNN," *Programming and Computer Software* 49, no. 8 (2023): 822–831, https://doi.org/10.1134/S0361768823080236.

64. N. A. Al-Saati and T. R. Alreffaee, "Software Effort Estimation Using Multi Expression Programming," *Al-Rafidain Journal of Computer Sciences and Mathematics* 11, no. 2 (2015): 53–71, https://doi.org/10.33899/csmj.2014.163756.

65. A. Ali and C. Gravino, "Improving Software Effort Estimation Using Bio-Inspired Algorithms to Select Relevant Features: An Empirical Study," *Science of Computer Programming* 205 (2021): 102621, https://doi.org/10.1016/j.scico.2021.102621.

66. I. U. Rehman, Z. Ali, and Z. Jan, "An Empirical Analysis on Software Development Efforts Estimation in Machine Learning Perspective," *ADCAIJ: Advances in Distributed Computing and Artificial Intelligence Journal* 10, no. 3 (2021): 227–240, https://doi.org/10.14201/ADCAIJ2021103227240.

67. W. Amaral, L. Rivero, G. B. Junior, and D. Viana, "Using Machine Learning Technique for Effort Estimation in Software Development," in *Proc. XVIII Brazilian Symp. Software Quality (SBQS)* (Association for Computing Machinery, 2019), 240–245, https://doi.org/10.1145/3364641.3364670.

68. A. Barcaui and A. Monat, "Who Is Better in Project Planning? Generative Artificial Intelligence or Project Managers?," *Project Leadership and Society* 4 (2023): 100101, https://doi.org/10.1016/j.plas.2023.100101.

69. I. Ben Kraiem, M. Ben Mabrouk, and L. De Jode, "A Comparative Study of Machine Learning Algorithm for Predicting Project Management Methodology," *Procedia Computer Science* 225 (2023): 665–675, https://doi.org/10.1016/j.procs.2023.10.052.

70. H. Chen, B. Xu, L. Z. R. Wong, and K. Zhong, "Enhancing Software Effort Estimation Through Reinforcement Learning-Based Project Management-Oriented Feature Selection," *International Journal of*

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Managing Projects in Business 18, no. 4–5 (2025): 733–759, https://doi.org/10.1108/IJMPB-03-2024-0065.

71. T. Javdani Gandomani, M. Dashti, H. Zulzalil, and A. B. Md Sultan, "Enhancing Software Effort Estimation in the Analogy-Based Approach Through the Combination of Regression Methods," *IEEE Access* 12 (2024): 152122–152137, https://doi.org/10.1109/ACCESS.2024.3480829.

72. M. R. Davahli, "The Last State of Artificial Intelligence in Project Management," arXiv preprint arXiv:2012.12262, 2020, https://doi.org/10.48550/arXiv.2012.12262.

73. V. S. Dave and K. Dutta, "Neural Network Based Models for Software Effort Estimation: A Review," *Artificial Intelligence Review* 42, no. 2 (2015): 295–307, https://doi.org/10.1007/s10462-012-9339-x.

74. H. D. P. de Carvalho, R. Fagundes, and W. Santos, "Extreme Learning Machine Applied to Software Development Effort Estimation," *IEEE Access* 9 (2021): 92676–92687, https://doi.org/10.1109/ACCESS.2021.3091313.

75. E. Ertugrul, Z. Baytar, C. Catal, and C. Muratli, "Performance Tuning for Machine Learning-Based Software Development Effort Prediction Models," *Turkish Journal of Electrical Engineering and Computer Sciences* 27, no. 2 (2019): 1308–1327, https://doi.org/10.3906/elk-1809-23.

76. E. M. De Bortoli Fávero, D. Casanova, and A. R. Pimentel, "SE^{3}M: A Model for Software Effort Estimation Using Pre-Trained Embedding Models," *Information and Software Technology* 147 (2022): 106886, https://doi.org/10.1016/j.infsof.2022.106886.

77. P. Jayadi, D. D. Prasetya, and T. Widiyaningtyas, "Explanation of a Simple Machine Learning Model for Software Effort Estimation Using SHAP (SHapley Additive exPlanations)," in *Proc. 9th Int. Conf. Electrical, Electronics and Information Engineering (ICEEIE)* (IEEE, 2025), 1–5, https://doi.org/10.1109/ICEEIE66203.2025.11254827.

78. M. S. Khan, F. Jabeen, S. Ghouzali, Z. Rehman, S. Naz, and W. Abdul, "Metaheuristic Algorithms in Optimizing Deep Neural Network Model for Software Effort Estimation," *IEEE Access* 9 (2021): 60309–60327, https://doi.org/10.1109/ACCESS.2021.3072380.

79. A. Kumar, "Recommendation of Machine Learning Techniques for Software Effort Estimation Using Multi-Criteria Decision Making," *Journal of Universal Computer Science* 30, no. 2 (2024): 221–241, https://doi.org/10.3897/jucs.110051.

80. S. Kumar, "Improved Methods for Software Fault Prediction and Web Service Selection," *CSI Transactions on ICT* 8, no. 1 (2020): 27–41, https://doi.org/10.1007/s40012-020-00284-6.

81. C. López-Martín, "Effort Prediction for the Software Project Construction Phase," *Journal of Software: Evolution and Process* 33, no. 7 (2021): e2365, https://doi.org/10.1002/smr.2365.

82. C. López-Martín and A. Abran, "Neural Networks for Predicting the Duration of New Software Projects," *Journal of Systems and Software* 101 (2015): 127–135, https://doi.org/10.1016/j.jss.2014.12.002.

83. Y. Mahmood, N. Kama, A. Azmi, A. S. Khan, and M. Ali, "Software Effort Estimation Accuracy Prediction of Machine Learning Techniques: A Systematic Performance Evaluation," *Software: Practice and Experience* 52, no. 1 (2022): 39–65, https://doi.org/10.1002/spe.3009.

84. P. Pandey and R. Litoriya, "Fuzzy Cognitive Mapping Analysis to Recommend Machine Learning-Based Effort Estimation Technique for Web Applications," *International Journal of Information Technology* 12 (2020): 121–127, https://doi.org/10.1007/s41870-020-00442-2.

85. A. G. PriyaVarshini, K. AnithaKumari, V. S. Bhalavaishnavi, C. Kalpana Devi, and S. Pranesh, "Machine Learning Approach for Software Effort Estimation Using Combination of Principal Component Regression and Neural Network," *Journal of Physics: Conference Series* 2325, no. 1 (2022): 012049, https://doi.org/10.1088/1742-6596/2325/1/012049.

86. M. Razi, A. K. Sharma, and Y. Rajput, "A Comparative Analysis of Different Machine Learning Techniques Used in Software Effort Estimation," in *Proc. 2nd Int. Conf. Computational Intelligence, Communication Technology and Networking (CICTN)* (IEEE, 2025), 385–393, https://doi.org/10.1109/CICTN64563.2025.10932574.

87. E. Rodríguez Sánchez, E. F. Vázquez Santacruz, and H. Cervantes Maceda, "Effort and Cost Estimation Using Decision Tree Techniques and Story Points in Agile Software Development," *Mathematics* 11, no. 6 (2023): 1477, https://doi.org/10.3390/math11061477.

88. S. M. Satapathy and S. K. Rath, "Empirical Assessment of Machine Learning Models for Effort Estimation of Web-Based Applications," in *Proc. 10th Innovations in Software Engineering Conference (ISEC)* (Association for Computing Machinery, 2017), 74–84, https://doi.org/10.1145/3021460.3021468.

89. S. M. Satapathy, B. P. Acharya, and S. K. Rath, "Class Point Approach for Software Effort Estimation Using Stochastic Gradient Boosting Technique," *ACM SIGSOFT Software Engineering Notes* 39, no. 3 (2015): 1–6, https://doi.org/10.1145/2597716.2597725.

90. S. M. Satapathy and S. K. Rath, "Empirical Assessment of Machine Learning Models for Agile Software Development Effort Estimation Using Story Points," *Innovations in Systems and Software Engineering* 13 (2017): 191–200, https://doi.org/10.1007/s11334-017-0288-z.

91. S. K. Sehra, Y. S. Brar, N. Kaur, and S. S. Sehra, "Software Effort Estimation Using FAHP and Weighted Kernel LSSVM Machine," *Soft Computing* 23, no. 21 (2019): 10881–10900, https://doi.org/10.1007/s00500-018-3639-2.

92. S. Sharma and S. Vijayvargiya, "Modeling of Software Project Effort Estimation: A Comparative Performance Evaluation of Optimized Soft Computing-Based Methods," *International Journal of Information Technology* 14, no. 5 (2022): 2487–2496, https://doi.org/10.1007/s41870-022-00962-5.

93. Y. Sheoraj and R. K. Sungkur, "Using AI to Develop a Framework to Prevent Employees From Missing Project Deadlines in Software Projects–Case Study of a Global Human Capital Management (HCM) Software Company," *Advances in Engineering Software* 170 (2022): 103143, https://doi.org/10.1016/j.advengsoft.2022.103143.

94. A. J. Sánchez-García, M. S. González-Hernández, K. Cortés-Verdín, and J. C. Pérez-Arriaga, "Software Estimation in the Design Stage With Statistical Models and Machine Learning: An Empirical Study," *Mathematics* 12, no. 7 (2024): 1058, https://doi.org/10.3390/math12071058.

95. S. Tariq, M. Usman, and A. C. M. Fong, "Selecting Best Predictors From Large Software Repositories for Highly Accurate Software Effort Estimation," *Journal of Software: Evolution and Process* 32, no. 10 (2020): e2271, https://doi.org/10.1002/smr.2271.

96. V. Uc-Cetina, "Recent Advances in Software Effort Estimation Using Machine Learning," arXiv preprint arXiv:2303.03482, 2023, https://doi.org/10.48550/arXiv.2303.03482.

97. A. Zakrani, H. Mustapha, and N. Abdelwahed, "Investigating the Use of Random Forest in Software Effort Estimation," *Procedia Computer Science* 148 (2019): 343–352, https://doi.org/10.1016/j.procs.2019.01.042.

98. T. İnan, T. Narbaev, and Ö. Hazır, "A Machine Learning Study to Enhance Project Cost Forecasting," *IFAC-PapersOnLine* 55, no. 10 (2022): 3286–3291, https://doi.org/10.1016/j.ifacol.2022.10.127.

99. B. K. Kumar, S. Bilgaiyan, and B. S. P. Mishra, "Software Effort Estimation Based on Ensemble Extreme Gradient Boosting Algorithm and Modified Jaya Optimization Algorithm," *International Journal of Computational Intelligence and Applications* 23, no. 1 (2024): 2350032, https://doi.org/10.1142/S1469026823500323.

100. K. H. Kumar and K. Srinivas, "Preliminary Performance Study of a Brief Review on Machine Learning Techniques for Analogy Based Software Effort Estimation," *Journal of Ambient Intelligence and Humanized Computing* 14 (2023): 2141–2165, https://doi.org/10.1007/s12652-021-03427-y.

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

101. Z. Mushtaq, S. Alshmrany, F. Alturise, and T. Alkhalifah, "Early Size and Effort Estimation of Mobile Application Development," *EAI Endorsed Transactions on Energy Web* 9, no. 37 (2022): e3, https://doi.org/10.4108/eai.31-5-2021.170010.

102. A. Bou Nassif, L. F. Capretz, and D. Ho, "Enhancing Use Case Points Estimation Method Using Soft Computing Techniques," arXiv preprint arXiv:1612.01078, 2016, https://doi.org/10.48550/arXiv.1612.01078.

103. P. Pospieszny, B. Czarnacka-Chrobot, and A. Kobylinski, "An Effective Approach for Software Project Effort and Duration Estimation With Machine Learning Algorithms," *Journal of Systems and Software* 137 (2018): 184–196, https://doi.org/10.1016/j.jss.2017.11.066.

104. E. Praynlin, "Using Meta-Cognitive Sequential Learning Neuro-Fuzzy Inference System to Estimate Software Development Effort," *Journal of Ambient Intelligence and Humanized Computing* 12, no. 9 (2021): 8763–8776, https://doi.org/10.1007/s12652-020-02652-1.

105. M. A. Al-Betar, S. Kassaymeh, S. N. Makhadmeh, S. Fraihat, and S. Abdullah, "Feedforward Neural Network-Based Augmented Salp Swarm Optimizer for Accurate Software Development Cost Forecasting," *Applied Soft Computing* 135 (2023): 111008, https://doi.org/10.1016/j.asoc.2023.111008.

106. T. R. Benala and R. Mall, "SEET: Software Development Effort Estimation Using Ensemble Techniques," *ACM SIGSOFT Software Engineering Notes* 43, no. 3 (2018): 17, https://doi.org/10.1145/3229783.3229805.

107. R. Bakhshi, S. F. Moradinia, R. Jani, and R. Vafaei Poor, "Presenting a Hybrid Scheme of Machine Learning Combined With Metaheuristic Optimizers for Predicting Final Cost and Time of Project," *KSCE Journal of Civil Engineering* 26, no. 8 (2022): 3188–3203, https://doi.org/10.1007/s12205-022-1424-3.

108. M. Dashti, T. Javdani Gandomani, D. Hasanpoor Adeh, H. Zulzalil, and A. B. Md Sultan, "LEMABE: A Novel Framework to Improve Analogy-Based Software Cost Estimation Using Learnable Evolution Model," *PeerJ Computer Science* 8 (2022): e800, https://doi.org/10.7717/peerj-cs.800.

109. J. K. Gardia, A. V. S. P. Kumar, and R. Nayak, "A Comprehensive Software Project Cost Assessment Framework Utilizing Global-Local Artificial Neural Networks Optimized With High-Level Target Navigation Pigeon-Inspired Optimization," in *Proc. 2025 4th OPJU International Technology Conference (OTCON) on Smart Computing for Innovation and Advancement in Industry 5.0* (IEEE, 2025), https://doi.org/10.1109/OTCON65728.2025.11070615.

110. A. Kaushik and N. Singal, "A Hybrid Model of Wavelet Neural Network and Metaheuristic Algorithm for Software Development Effort Estimation," *International Journal of Information Technology* 14, no. 3 (2022): 1689–1698, https://doi.org/10.1007/s41870-019-00339-1.

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Appendix A

TABLE A1 | Quality assessment scores.

ID	Citation label (author, year)	Article title	Quality assessment score
1	Abdulmajeed et al., 2021	Predict the Required Cost to Develop Software Engineering Projects by Using Machine Learning	0.7
2	Abnane et al., 2017	Empirical Evaluation of Fuzzy Analogy for Software Development Effort Estimation	0.7
3	Abnane et al., 2021	Heterogeneous Ensemble Imputation for Software Development Effort Estimation	0.8
4	Al-Betar et al., 2023	Feedforward Neural Network-Based Augmented Salp Swarm Optimizer for Accurate Software Development Cost Forecasting	0.4
5	Al-Saati et al., 2015	Software Effort Estimation Using Multi Expression Programming	0.8
6	Naif Alatawi et al., 2024	Forecasting the Software Engineering Model's Effort Estimation Using Constructive Cost Estimation Models	0.7
7	A. Ali et al., 2019	A Systematic Literature Review of Software Effort Prediction Using Machine Learning Methods	0.6
8	A. Ali et al., 2021	Improving Software Effort Estimation Using Bio-Inspired Algorithms to Select Relevant Features: An Empirical Study	0.8
9	I. Ur Rehman et al., 2021	An Empirical Analysis on Software Development Efforts Estimation in Machine Learning Perspective	0.9
10	Amaral et al., 2019	Using Machine Learning Technique for Effort Estimation in Software Development	0.7
11	Amazal et al., 2015	Software Development Effort Estimation Using Classical and Fuzzy Analogy: A Cross-Validation Comparative Study	0.5
12	CH Anitha et al., 2024	Deep Artificial Neural Network Based Multilayer Gated Recurrent Model for Effective Prediction of Software Development Effort	0.8
13	Hajar Arbain et al., 2019	Adoption of Machine Learning Techniques in Software Effort Estimation: An Overview	0.6
14	Mohit Arora et al., 2022	An Efficient ANFIS-EEBAT Approach to Estimate Effort of Scrum Projects	0.8
15	Attarzadeh et al., 2015	Proposing an Effective Artificial Neural Network Architecture to Improve the Precision of Software Cost Estimation Model	0.8
16	Azath et al., 2020	Software Effort Estimation Using Modified Fuzzy C Means Clustering and Hybrid ABC-MCS Optimization in Neural Network	0.6
17	Mohammad Azzeh et al., 2023	Software Effort Estimation Using Convolutional Neural Network and Fuzzy Clustering	0.9
18	Azzeh et al., 2017	Fuzzy Model Tree For Early Effort Estimation	0.8
19	Barcaui et al., 2023	Who Is Better in Project Planning? Generative Artificial Intelligence or Project Managers?	0.9
20	Ines BEN et al., 2023	A Comparative Study of Machine Learning Algorithm for Predicting Project Management Methodology	0.5
21	Benala et al., 2018	SEET: Software Development Effort Estimation Using Ensemble Techniques	0.8
22	Chen et al., 2024	Enhancing Software Effort Estimation through Reinforcement Learning-Based Project Management-Oriented Feature Selection	0.8
23	Chhabra et al., 2020	Optimizing Design of Fuzzy Model for Software Cost Estimation Using Particle Swarm Optimization Algorithm	0.7
24	Bakhshi et al., 2022	Presenting a Hybrid Scheme of Machine Learning Combined With Metaheuristic Optimizers for Predicting Final Cost and Time of Project	0.4
25	Dashti et al., 2024	Enhancing Software Effort Estimation in the Analogy-Based Approach Through the Combination of Regression Methods	0.8

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

ID	Citation label (author, year)	Article title	Quality assessment score
26	Dashti et al., 2022	LEMABE: A Novel Framework To Improve Analogy-Based Software Cost Estimation Using Learnable Evolution Model	0.9
27	Davahli et al., 2020	The Last State of Artificial Intelligence in Project Management	0.5
28	Dave et al., 2015	Neural Network Based Models For Software Effort Estimation: A Review	0.7
29	De Carvalho et al., 2021	Extreme Learning Machine Applied to Software Development Effort Estimation	0.7
30	Moatasem. M. Draz et al., 2024	Software Cost Estimation Prediction Using a Convolutional Neural Network and Particle Swarm Optimization Algorithm	0.8
31	Wei Lin Du and Capretz et al., 2015	A Hybrid Intelligent Model for Software Cost Estimation	0.8
32	Wei Lin Du and Ho et al., 2015	Improving Software Effort Estimation Using Neuro-Fuzzy Model With SEER-SEM	0.8
33	Omar El Beggar et al., 2023	IFEJM: New Intuitionistic Fuzzy Expert Judgment Method for Effort Estimation in Agile Software Development	0.8
34	Ertugrul et al., 2019	Performance Tuning for Machine Learning-Based Software Development Effort Prediction Models	0.9
35	Ferreira et al., 2015	Artificial Intelligence Techniques for Estimating the Effort in Software Development Projects	0.9
36	Ferreira et al., 2015	Effort Estimation of Software Projects Using Artificial Intelligence Techniques	0.9
37	Eliane M De Bortoli et al., 2020	SE^3M: A Model for Software Effort Estimation Using Pre-trained Embedding Models	0.8
38	Jitendra Kumar et al., 2025	A Comprehensive Software Project Cost Assessment Framework utilizing Global-Local Artificial Neural Networks Optimized With High-Level Target Navigation Pigeon-Inspired Optimization	0.9
39	Sunil Kumar Gouda et al., 2022	Software Cost Estimation Model Based on Fuzzy C-Means and Improved Self Adaptive Differential Evolution Algorithm	0.8
40	Gultekin et al., 2020	Story Point-Based Effort Estimation Model With Machine Learning Techniques	0.6
41	Hameed et al., 2023	An Optimized Case-Based Software Project Effort Estimation Using Genetic Algorithm	0.8
42	Iordan et al., 2024	An Optimized LSTM Neural Network for Accurate Estimation of Software Development Effort	0.8
43	Puguh Jayadi et al., 2025	Explanation of a Simple Machine Learning Model for Software Effort Estimation Using SHAP (SHapley Additive exPlanations)	0.8
44	Sofian Kassaymeh et al., 2022	Software Effort Estimation Modeling and Fully Connected Artificial Neural Network Optimization Using Soft Computing Techniques	0.9
45	Ishleen Kaur et al., 2018	Neuro Fuzzy COCOMO II Model for Software Cost Estimation	0.8
46	Anupama Kaushik et al., 2023	SENSE: Software Effort Estimation Using Novel Stacking Ensemble Learning	0.8
47	Anupama Kaushik et al., 2015	An Improved Functional Link Artificial Neural Networks With Intuitionistic Fuzzy Clustering for Software Cost Estimation	0.8
48	Kaushik et al., 2022	A Hybrid Model of Wavelet Neural Network and Metaheuristic Algorithm for Software Development Effort Estimation	0.7
49	Khan et al., 2021	Metaheuristic Algorithms in Optimizing Deep Neural Network Model for Software Effort Estimation	0.7
50	Kumar et al., 2024	Recommendation of Machine Learning Techniques for Software Effort Estimation Using Multi-Criteria Decision Making	0.6
51	Kumar et al., 2024	Software Effort Estimation Based on Ensemble Extreme Gradient Boosting Algorithm and Modified Jaya Optimization Algorithm	0.6
52	K. Harish Kumar et al., 2021	Preliminary Performance Study of a Brief Review on Machine Learning Techniques for Analogy Based Software Effort Estimation	0.8
53	K. Harish Kumar et al., 2024	An Improved Analogy-Rule Based Software Effort Estimation Using HTRR-RNN in Software Project Management	0.9

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

ID	Citation label (author, year)	Article title	Quality assessment score
54	K. Harish Kumar et al., 2023	An Accurate Analogy Based Software Effort Estimation Using Hybrid Optimization and Machine Learning Techniques	0.8
55	Kumar et al., 2020	Improved Methods for Software Fault Prediction and Web Service Selection	0.7
56	Lalitha et al., 2023	A Methodology to Analyze and Estimate the Software Development Process Using Machine Learning Techniques	0.8
57	Safae Laqrichi et al., 2015	Integrating Uncertainty in Software Effort Estimation Using Bootstrap-Based Neural Networks	0.6
58	López-Martín et al., 2021	Effort Prediction for the Software Project Construction Phase	0.8
59	López-Martín et al., 2015	Predictive Accuracy Comparison Between Neural Networks and Statistical Regression for Development Effort of Software Projects	0.7
60	López-Martín et al., 2015	Neural Networks for Predicting the Duration of New Software Projects	0.8
61	Mahdi et al., 2021	Software Project Management Using Machine Learning Technique: A Review	0.8
62	Mahmood et al., 2020	A Systematic Review of Studies on Use Case Points and Expert-Based Estimation of Software Development Effort	0.6
63	Mahmood et al., 2021	Software Effort Estimation Accuracy Prediction of Machine Learning Techniques: A Systematic Performance Evaluation	0.8
64	Miandoab et al., 2016	A Novel Hybrid Algorithm for Software Cost Estimation Based on Cuckoo Optimization and K-Nearest Neighbors Algorithms	0.7
65	Mushtaq et al., 2022	Early Size and Effort Estimation of Mobile Application Development	0.4
66	Ali Bou Nassif et al., 2016	Neural Network Models for Software Development Effort Estimation: A Comparative Study	0.8
67	Ali Bou Nassif and Azzeh et al., 2019	Software Development Effort Estimation Using Regression Fuzzy Models	0.8
68	Ali Bou Nassif and Capretz et al., 2016	Enhancing Use Case Points Estimation Method Using Soft Computing Techniques	0.8
69	Pandey et al., 2020	Fuzzy Cognitive Mapping Analysis to Recommend Machine Learning-Based Effort Estimation Technique for Web Applications	0.9
70	Phan et al., 2022	Heterogeneous Graph Neural Networks for Software Effort Estimation	0.8
71	Pospieszny et al., 2018	An Effective Approach for Software Project Effort and Duration Estimation With Machine Learning Algorithms	0.5
72	Praynlin et al., 2021	Using Meta-Cognitive Sequential Learning Neuro-Fuzzy Inference System to Estimate Software Development Effort	0.8
73	Predescu et al., 2019	Software Effort Estimation Using Multilayer Perceptron and Long Short Term Memory	0.8
74	Priya Varshini et al., 2022	Machine Learning Approach for Software Effort Estimation Using Combination of Principal Component Regression and Neural Network	0.7
75	Priya Varshini et al., 2024	Optimized Convolutional Neural Network Model for Software Effort Estimation	0.9
76	Yashendra Rajput et al., 2025	A Comparative Analysis of Different Machine Learning Techniques Used in Software Effort Estimation	0.9
77	Dragica Rankovic et al., 2021	Convergence Rate of Artificial Neural Networks for Estimation in Software Development Projects	0.6
78	Nevena Rankovi et al., 2024	Interpretable Software Estimation With Graph Neural Networks and Orthogonal Array Tuning Method	0.6
79	Rankovic et al., 2021	A New Approach to Software Effort Estimation Using Different Artificial Neural Network Architectures and Taguchi Orthogonal Arrays	0.7
80	Rankovic et al., 2022	COSMIC FP Method in Software Development Estimation Using Artificial Neural Networks Based on Orthogonal Arrays	0.8
81	Rankovic et al., 2021	Improved Effort and Cost Estimation Model Using Artificial Neural Networks and Taguchi Method With Different Activation Functions	0.7

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

ID	Citation label (author, year)	Article title	Quality assessment score
82	Rhmann et al., 2022	Software Effort Estimation Using Ensemble of Hybrid Search-Based Algorithms Based on Metaheuristic Algorithms	0.8
83	Rodríguez Sánchez et al., 2023	Effort and Cost Estimation Using Decision Tree Techniques and Story Points in Agile Software Development	0.6
84	Zaineb Sakhrawi et al., 2021	Software Enhancement Effort Prediction Using Machine-Learning Techniques: A Systematic Mapping Study	0.7
85	Sanchez-Garcia et al., 2023	Gradient Boosting Optimized Through Differential Evolution for Predicting the Testing Effort of Software Projects	0.6
86	Satapathy et al., 2017	Empirical Assessment of Machine Learning Models for Effort Estimation of Web-Based Applications	0.9
87	Satapathy et al., 2015	Class Point Approach for Software Effort Estimation Using Stochastic Gradient Boosting Technique	0.7
88	Satapathy et al., 2017	Empirical Assessment of Machine Learning Models for Agile Software Development Effort Estimation Using Story Points	0.7
89	Sehra et al., 2019	Software Effort Estimation Using FAHP and Weighted Kernel LSSVM Machine	0.9
90	Archana Sharma et al., 2023	A Memetic Approach for Optimizing Software Effort Estimation Using Anti-Predatory NIA	0.9
91	Sharma et al., 2022	Analysis of Software Effort Estimation Based on Story Point and Lines of Code Using Machine Learning	0.6
92	Sudhir Sharma et al., 2022	Modeling of Software Project Effort Estimation: A Comparative Performance Evaluation of Optimized Soft Computing-Based Methods	0.8
93	Yugeshwaree Sheoraj and Roopesh Kevin Sungkur et al., 2022	Using AI to Develop a Framework to Prevent Employees From Missing Project Deadlines in Software Projects: Case Study of a Global Human Capital Management (HCM) Software Company	0.4
94	Silhavy et al., 2016	Algorithmic Optimization Method for Effort Estimation	0.8
95	Singh et al., 2020	Cost Estimation Model Using Enhance-Based Differential Evolution Algorithm	0.8
96	Sousa et al., 2023	Applying Machine Learning to Estimate the Effort and Duration of Individual Tasks in Software Projects	0.8
97	Sree P et al., 2016	Analytical Structure of a Fuzzy Logic Controller for Software Development Effort Estimation	0.3
98	Sánchez-García et al., 2024	Software Estimation in the Design Stage With Statistical Models and Machine Learning: An Empirical Study	0.8
99	Tariq et al., 2020	Selecting Best Predictors From Large Software Repositories for Highly Accurate Software Effort Estimation	0.6
100	Uc-Cetina et al., 2023	Recent Advances in Software Effort Estimation Using Machine Learning	0.8
101	Valdés-Souto et al., 2023	Improving a Model for NFR Estimation Using Band Classification and Selection With KNN	0.8
102	Priya Varshini et al., 2021	Comparative Analysis of Machine Learning and Deep Learning Algorithms for Software Effort Estimation	0.9
103	Velarde et al., 2016	Analyzing the Effect of Variables in the Software Development Effort Estimation	0.2
104	Zakrani Abdelali et al., 2019	Investigating the Use of Random Forest in Software Effort Estimation	0.6
105	Zhang et al., 2021	Efficiency Improvement of Function Point-Based Software Size Estimation With Deep Learning Model	0.7
106	Öztürk et al., 2022	A Tuned Feed-Forward Deep Neural Network Algorithm for Effort Estimation	0.8
107	Tolga İnan et al., 2022	A Machine Learning Study to Enhance Project Cost Forecasting	0.6
108	Şengüneş et al., 2023	An Artificial Neural Network Model for Project Effort Estimation	0.8

20477481, 2026, 8, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/smr.70158, Wiley Online Library on [09/10/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License